Engineering the Software Stack of a Highly Automated Robot in the FIRST Robotics Competition

We are creating a Robot for 2026 FIRST Robotics Competition as team **#8242** *TeknoNFL*
In this paper I present the development of highly customized Robot software stack, including the design decisions in our architecture(software focused but including mechanical decisions too) , our overall development process (autonomous and shooting automating the shooter for teleop with a digital-twin algorithm (field localization system) , and how we solved low-cost problems by creating alternative electronic components etc.
Almost the whole process of deciding the software structure (actually the overall robot design process) **depends on the** seasons *game* . This information looks simple but this means that The software structure (and of course developing roadmap) is nearly all depends on the tactical analysis and a little bit of the customisation for driver.

Because of the preferences for the Robots **depends on** the game which every year changes, for explaining the preferences clearly i will also talk about the tactical analysis of the game and the game field.
in a nutshell:
This paper is not specified for 2026 game but i will give examples from this year for explaining this with a really-subject. This paper aims to give a perspective to the reader for making the development process **proactive**.

Note: We are currently developing a framework named ***FIRSTMOVE***, it is an FRC framework designed for strategic team, match, and tactic analysis. It processes raw match and team data and evaluates performance using the M.O.V.E (Match-Oriented Value Evaluator) module. The system supports the drive team by enabling proactive, data-driven alliance strategies and advanced analyses such as scouting, performance enumeration, playstyle pattern analysis, and alliance tactic cross-checking. The framework supports customization and module development for sophisticated tasks. The goal is simple: make the right first move.
This "subStack" is not directly effect the robots performance. But the true strategy, with a good driver, could you wins a match even against a difficult opponent (technically more advance than our alliance).
We will talk about FIRSTMOVE in later.

"Firstly" lets begin with FIRST ROBOTICS COMPETITION

FIRST ROBOTICS COMPETITION

FIRST Robotics Competition (FRC) is often described as a “robot competition,” but that definition is too small. Yes, teams build a robot and compete on a field—however the real value is the *engineering process* and the *team process* around it. FRC is designed to prepare high school students for engineering life and professional work culture: building under deadlines, managing requirements, documenting decisions, testing under real constraints, and iterating fast when something breaks.

Every season comes with a new game, new rules, and a new field layout. That yearly change is not a “detail”; it forces teams to think like real engineers. You don’t build the same product every year—you analyze a new problem, read the manual like a specification document, extract constraints, then make trade-offs between performance, reliability, cost, and time. In that sense, the FRC field is basically a

controlled environment that simulates industrial systems: moving mechanisms, sensing, networking (CAN, control loops, vision), safety requirements, inspection rules, and failure modes that you only discover when you test with real hardware.

That is why FRC is more than robotics. It is a full-stack project: mechanical design, electronics, embedded systems, software architecture, data analysis, strategy, and operations. You also learn how engineering is never “only technical.” A robot can be great on paper and still lose if the team cannot coordinate, communicate, plan, and execute consistently. The competition forces you to collaborate under pressure, explain your system to others, and make decisions quickly with limited information—exactly like real work life.

On a personal level, FRC has had a direct impact on me too. Through this competition, my human relationships improved, and my leadership skills developed. Being in a team, organizing work, keeping people aligned, handling stress, and solving conflicts without breaking motivation—these are skills you don’t learn from tutorials. You learn them because you *have to* if you want the robot (and the team) to survive the season.

Finally, one of the core cultural principles of FIRST is **Gracious Professionalism**. In short: compete hard, but treat others with respect; help teams even if they are your opponents; share knowledge; and keep the community healthy. This culture matters because it turns the event into a learning ecosystem. You are not only trying to win matches—you are also building long-term engineering habits and professional behavior. That mindset is one of the reasons FRC creates strong engineers *and* strong people.

The FRC Process of Team #8242

Team#*8242*: *TeknoNFL*

In this section i will introduce my team. This may look non-technical, but it directly shaped our engineering constraints and trade-offs. Although it may not seem that important, the relationship between the robot building process and the team's situation is significant.
In this section i will introduces my team. Although this information may seem non-technical, the relationship between the robot development process and the team’s structure is significant and should not be overlooked.

The purpose of providing this context is to better visualize our capabilities and working conditions. Many of our design decisions were shaped by limited resources and the need to develop low-cost alternative solutions.

The team’s structure, internal dynamics, and management approach directly affect the development process and engineering outcomes; as a team captain, I recognize that leadership, communication, coordination, and effective team management play a critical role in engineering efficiency and decision-making quality.

However, since this is a technical paper, topics related to team management and leadership are out of scope but i will write a paper for this topic later.

Beginning

The team was founded in 2018 at Niğde Science High School under the name TeknoNFL. Our FRC journey started in 2020.

Discontinuation

After our rookie year, the COVID-19 pandemic began. Following that period, we did not participate in any FIRST events for five years.

Other STEM Activities

During this time, the team did not disappear. We participated in Teknofest (UAV category), MEB Robot competitions, TÜBİTAK projects, and CTF cybersecurity competitions. Even though we were away from FRC, the engineering culture continued.

Rebirth of the FRC Team – Five Years Later

Our team was returning from a MEB Robot competition when one of our members — who had close contact with a former mentor of #8242 — talked about FRC during the bus ride. As he described the scale and philosophy of the competition, the whole team became determined to join.

Five years after our rookie season, we returned to FRC — and were honored with the **Rising All-Star Award**.

Financial Situation

Almost every major decision about the robot was influenced by our financial condition.

General

This was not unique to FRC; in other competitions as well, we often had to balance ambition with budget.

This Year Specific

This season, financial limitations directly shaped our hardware choices and forced us to design alternative solutions instead of purchasing expensive components.

Team Structure

We are mostly 11th-grade students. Because of this, academic pressure and exam preparation create an additional challenge. Despite that, we tried to maintain balance and stay committed to the process.

The Software Team

Our software team consists of three members: one rookie and two veterans. Even with financial disadvantages, we aimed to compensate through engineering effort. When necessary, we designed and manufactured components ourselves. On the software side, we developed customized algorithms to close the gap created by limited hardware resources.

Before 2026 Season Kick-Off

Before Kick-Off, the team was tired. Motivation was low, and processes were progressing slower than they should have. The previous months had drained our energy.

After Kick-Off

After Kick-Off, everything changed. We reminded ourselves that if we chose to enter this competition, we had to give our full effort. Since most of us are in 11th grade, this may be our last FRC season.

That realization became a strong source of motivation.

To avoid losing the scope, we are done with teams structure but later, i am planning to writing a paper about leadership and management for FRC.

Realization for tactical improvements

Looking at this season's game design, it feels like it borrows a lot of its "core tension" from what we saw in the 2025 FRC season: matches where gaining advantage wasn't only about scoring faster, but also about controlling access to the game pieces and disrupting the opponent's cycle. When the game piece is standardized and contact is basically unavoidable, the match becomes less about "who has the fanciest mechanism" and more about **who controls the flow of the field**.

The fact that hubs can switch between active and inactive states, and that extra fuel can appear in regions like outposts or depots, pushes teams into building flexible cycle plans. In other words, a good alliance isn't locked into one pattern — it can adapt mid-match depending on what is available, what the opponent is doing, and which zones are currently valuable. That naturally encourages dynamic role distribution: one robot might run clean scoring cycles, another can play denial or disruption, and another can pivot depending on the field state.

Because close interaction with opponents is built into the game and the material is essentially "contestable," field positioning becomes a serious skill. Timing, lane control, and when to engage physically start to matter as much as raw scoring speed. If you hesitate, you lose the piece; if you take the wrong route, you waste seconds; and in a game where many robots end up having similar operational capacity, those seconds decide matches.

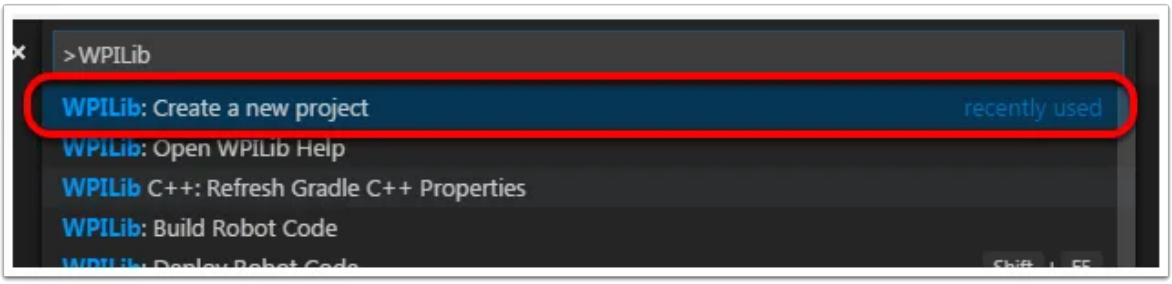
That's the other big point: compared to seasons that forced highly complex engineering solutions, this year's task set seems more streamlined. With fewer "extreme" design requirements, you get a field where lots of robots can reach "good enough." When that happens, the match outcome shifts toward alliance coordination — who does what, when, and why — and whether your team can build a counter-plan against the opponent's style.

So the deciding factors become very tactical: correct role assignment inside the alliance, real coordination instead of three robots doing the same thing, strong pre-match preparation, and scouting that's actually usable. At that point, winning isn't strictly tied to having the most advanced hardware — it's tied to making fewer wrong decisions than the other side, and executing a plan that fits the reality of the match.

Development Process

The setup preference

When you create project in "Wpilib vscode", you should select a project type.



When you create a WPILib project, the first "architecture decision" happens before writing any real code: **TimedRobot** or **Command-Based**.

For beginners, **TimedRobot is recommended** because it is simple: you write code inside lifecycle functions like `robotInit()`, `teleopPeriodic()`, `autonomousPeriodic()` and it runs at ~50Hz. There is no extra structure to learn, no scheduler, no command patterns. You can bring motors alive quickly and understand the control flow easily.

However, as the robot grows (multiple mechanisms, automated sequences, sensor fusion, vision, safety interlocks), most advanced teams migrate to **Command-Based** because it scales better and forces a clean structure.

https://www.chiefdelphi.com/t/timed-vs-command-based/418622
https://docs.wpilib.org/en/stable/docs/software/examples-tutorials/wpilib-examples.html#wpilib-example-projects (Look this to analyze example codes)

Differential Drive

Actually developing a drive code for FRC is quite easy and especially for tankdrive (we are using tankdrive). There is so much information for this so i don't dive deeper but i think we followed a *different path than usual*.

2025 drive_code

This years was my first year at FRC, so it was a learning year but i make a suggestion list:

*__!Lesson learned:__ from real life participation

- Always backup your code; not just for FRC, backup everything and writing a documentation is good if you have time and energy. We lost the source code from 2025 and we
- You can start and customize the drive code before kick-off. And yes you should customize the code for the driver for better performance.
- Read the game manual and look at the *inspection checklist*; you should update your game tools and some firmwares. If you don't you can't play and updating sometimes cause dependency-hell, especially if you use old devices like VictorSPX. We didn't update the game tools in 2025 and that cost us time and panic-increasing in the competition
- You need time to optimize and test your code. Particularly If mechanical build time delay (in our case, the build is contuined even before the competition day). Testing the code and mechanical subsystems takes time. You should done for mechanical about two weeks before the competition and done the autonomous 1 weeks before the competition. Debugging and optimizing takes time. This situation teached us the importance of time managing. Also don't forget
- Don't forget the joystick. Yes actually we forgot. Nearly 800km away. :D
- Look at other teams codes from past years and use AI consciously. We did't use AI and some task took time. But we started early so it was not problem and just looking at the documentations for writing code improved the software team which had 2 member :D
  We were use VictorSPX and newer CTRE Phoenix versions (Phoenix 6/Pro) won’t support them directly and To program Victor SPX in 2025/2026, you typically must include **Phoenix 5** libraries alongside your WPILib build so you should manage the dependency. And it was a little frustrating in our case.

  In our FRC season, we implemented the robot motion control logic using explicit conditional structures (e.g., handling different axis combinations with detailed if–else rules), despite the

availability of existing motion-control libraries. This approach allowed us to fully understand how it works.

2026 Drive_Code

https://github.com/TeknoNFL-8242/2026RobotCodeFRC-

Logarithmic Drive (Input Shaping) — why we wrote our own curve

In FRC, drivetrain code is often treated as “solved” because WPILib already provides arcade/curvature drive utilities. However, in real matches, *driver feel* is not a minor detail — it is directly connected to cycle time, aiming stability, and collision recovery. Our 2026 drivetrain control is built on a custom input-shaping method that uses a **logarithmic response curve** for turning and a **speed-aware steering limiter** for stability.

The core problem is simple:

- **Linear joystick mapping** makes the robot *too sensitive* near the center (small stick movement → too much turning). (this interpretation was incorrect. > look at the *Realization from 6 month later(08/09/2026)* title)
- If we reduce sensitivity with a big deadband, we lose fine control.
- If we keep it linear, the robot becomes “twitchy,” especially while shooting alignment and tight lane driving.
  So instead of adding random “driver preferences” manually, we encoded it into math.

The curve: log-scaled steering factor `f(x)`

We calculate an “effective turn weight” from the joystick’s horizontal axis:

$$f(x) = \frac{\ln(1 + k|x|)}{\ln(1 + k)}$$

In code:

```
double xAbs = Math.abs(x_ax);
double f = Math.log(1.0 + k * xAbs) / LOG_K;
```

Where:

- `x` is joystick steering input (0..1),
- `k` is the aggressiveness factor,
- `LOG_K = ln(1+k)` normalizes the curve so **f(1)=1**.

Behavior:

- Near he center, the curve grows slowly → micro-corrections become possible.
- At large stick values, the curve rises faster → driver still has full steering authority.
  This gives us “precision when we need it” without sacrificing max turn response.

Realization from 6 month later(08/09/2026): Today, randomly decided to look at the github upload some files. Then I looked at this unfinished documentation. I want to make this documentation realy comprehensive about the whole process of the developement of the software. But because of the time limitation, even the most crucial work of arts are not finished in real life. And writed this documentation in a chaotic, time limited moment too.

Whatever, today I realise that f(x) function is bullshit and made drive-train unreliable to control.
Why concave funcs is not what we wanted: Because it makes turnings agrassive for lower X_axix values. But we need to make turning less agrasive for lower X_axix values and increase agrassivity for bigger values.
Before that func we used Linear equation, but this function makes controlling probably even worse. We need a convex function for this job.

$$f(x) = a \cdot x^3 + (1 - a) \cdot x$$

at

$$0 \leq a \leq 1 \qquad and \qquad 0 <\mid x \mid< 1$$

This function is a reasonable candidate for what we wanted: **less aggressive response around small X-axis inputs and increasingly stronger response toward larger inputs.

Why I made this mistake? Actually while I was driving the robot, I realized that the steering felt agrassive. However I assumed that perhaps the coefficient values simply were not tuned correctly, because we didn't have enough time to test. Also I think that this robot is unreliable because drivetrains are not best devices to control. But this assumptions are not necessarily correct. The problem was at the functions.
I don't remember why I think for this algortihm I need concave function but I have a few estimation:

1. I looked just f(x) func graph but this func does not dirrectl gives the motor output. I need to check the graph of final output after mixing(Disabling g(y) and looking just the behaivor of the effect of f(x) to motor output probably give me a clear notion)
2. I fly FPV drones and use FPV simulators. The first urge that came to me while I was looking at the rate-profile settings. I could somehow misinterpretate the graphs. I don't remember exactly what I was thinking when I originally designed the logarithmic function, so this remains only a hypothesis. But still, we need to test both algorithms to draw a definitive conclusion.
   Btw g(y) works okay, no problem with that

Speed-aware steering limiter `g(y)`

Even with a good curve, turning at high speed can cause instability (skidding, over-rotation, driver fighting the robot). To prevent that, we scale steering by forward speed:

$$g(y) = (1 - \mid y \mid)^a$$

$y \in [-1, 1]$: y equals to joysticks y_axis

In code:

```
double yAbs = Math.abs(y_ax);
double g = Math.pow(1.0 - yAbs, a);
```

Where:

- `y` is forward/back input,
- `a` controls how hard steering is “suppressed” at speed.
  Interpretation:
- If the robot is almost stopped (`|y|≈0`) → `g≈1` → steering is fully enabled.
- If the robot is near full speed (`|y|≈1`) → `g≈0` → steering is damped, robot holds lane.

This is basically “lane stability” encoded into math.

Final differential-drive mixing

We apply these terms when both X and Y are active (driver is trying to drive and steer simultaneously). The idea is to keep forward power mostly intact and inject controlled differential steering:
example:
leftPower = yAbs + yAbs *f* g;
rightPower = yAbs * (1.0 - f);

Then we restore signs based on quadrant (forward/back + left/right), so the robot behaves consistently in all directions.

```
x > 0, y > 0      Forward + Right
x < 0, y > 0      Forward + Left
x > 0, y < 0      Backward + Left
x < 0, y < 0      Backward + Right
```

The equations are actually mirrored versions of each other.

Forward + Right

```
leftPower  = yAbs + yAbs * f * g;
rightPower = yAbs * (1.0 - f);
```

Forward + Left

```
leftPower  = yAbs * (1.0 - f);
rightPower = yAbs + yAbs * f * g;
```

Backward + Left

```
leftPower  = -yAbs * (1.0 - f);
rightPower = -yAbs - yAbs * f * g;
```

Backward + Right

```
leftPower  = -yAbs - yAbs * f * g;
rightPower = -yAbs * (1.0 - f);
```

Quadrant separations make sure that if the plot works correctly at `(0,1)`, we can just mirror it to get the correct behavior in the other quadrants.

Result: a drivetrain that is:

- stable at speed,
- sensitive for small alignment,
- still aggressive when the driver commits to a hard turn.

4) Coefficients we used (and how we tuned them)

We used:

- `k = 1.8` (log curve aggressiveness)
- `a = 3.9` (steering suppression exponent)

Practical tuning logic:

- If the robot feels “nervous” in micro-steering → decrease `k`.
- If the robot cannot turn enough at moderate stick input → increase `k`.
- If the robot over-rotates while driving fast → increase `a`.
- If the robot feels “too locked straight” at speed → decrease `a`.

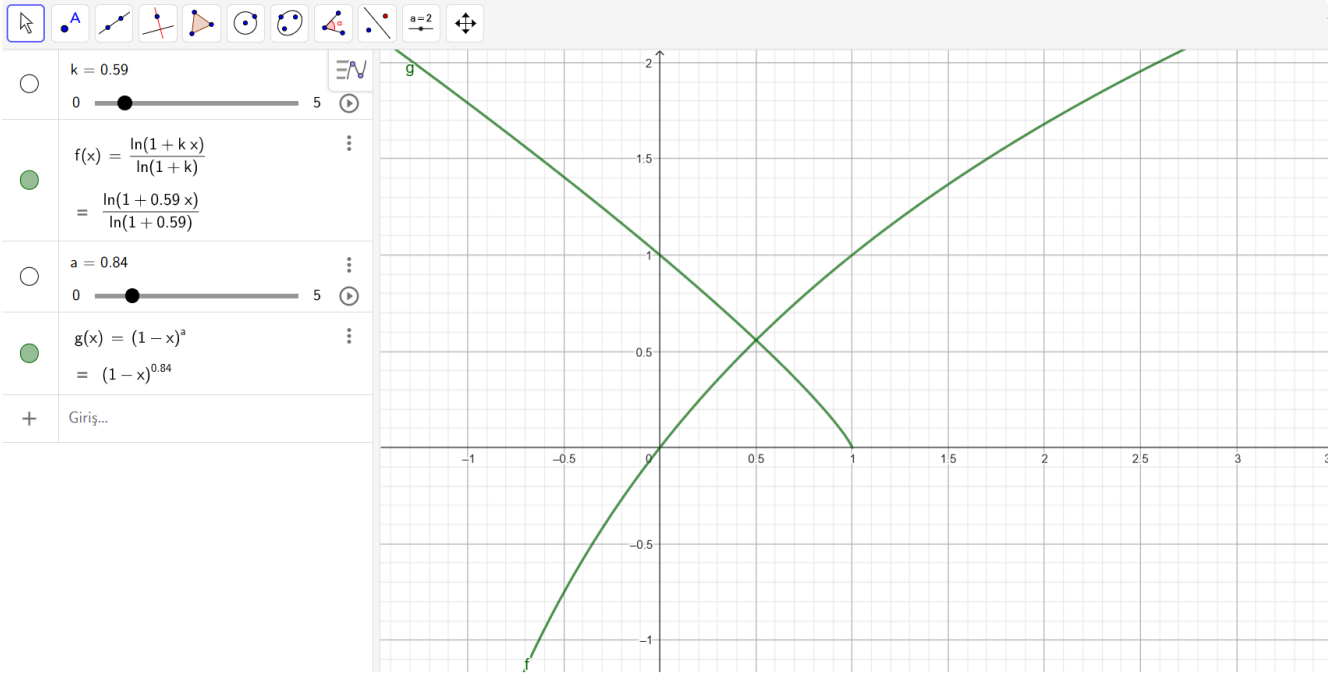
The tuning process was supported by our **Terminal Dashboard**, where we simulated joystick input and observed motor outputs before testing on the real robot. Also we wrote Terminal-Dahboard module that calculate the outputs and help me for tunning the drivetrain

5) Deadzone & the “Z-Lock” safety

For stability and predictable driver behavior, we also implemented:

- **deadzone** to remove stick noise,
- **zLock** to prevent accidental spin while the driver is already translating.
  This avoids the common problem where the rotation axis (Z) starts fighting the X/Y drive intent.
  In short:
- If driver is translating (x or y not centered) → lock out pure spin unless intentionally requested.
- If all centered → unlock spin.
  This increased repeatability a lot, especially in traffic and while lining up shots.

Lesson learned (driver-side reality): A drivetrain can be “technically correct” and still be bad if it is not *drivable under stress*.
This custom curve was one of the simplest changes that produced one of the biggest improvements in match consistency. It also made driver training more efficient: the robot



The Driver

Advice: Selecting the driver from the software team can create a serious advantage. Driving is not only about reflexes; it is also about understanding the system you are controlling. When the driver is involved in the codebase, they don't just "use" the robot — they understand its behavior at a structural level.

A software-oriented driver can directly customize the drive logic according to personal control preference. Deadbands, response curves, logarithmic scaling, ramp limits, rotation locks — these are not abstract settings. They shape how the robot feels in motion. If the driver writes or deeply understands the code, adjusting these parameters becomes a rational engineering process rather than trial-and-error guessing.

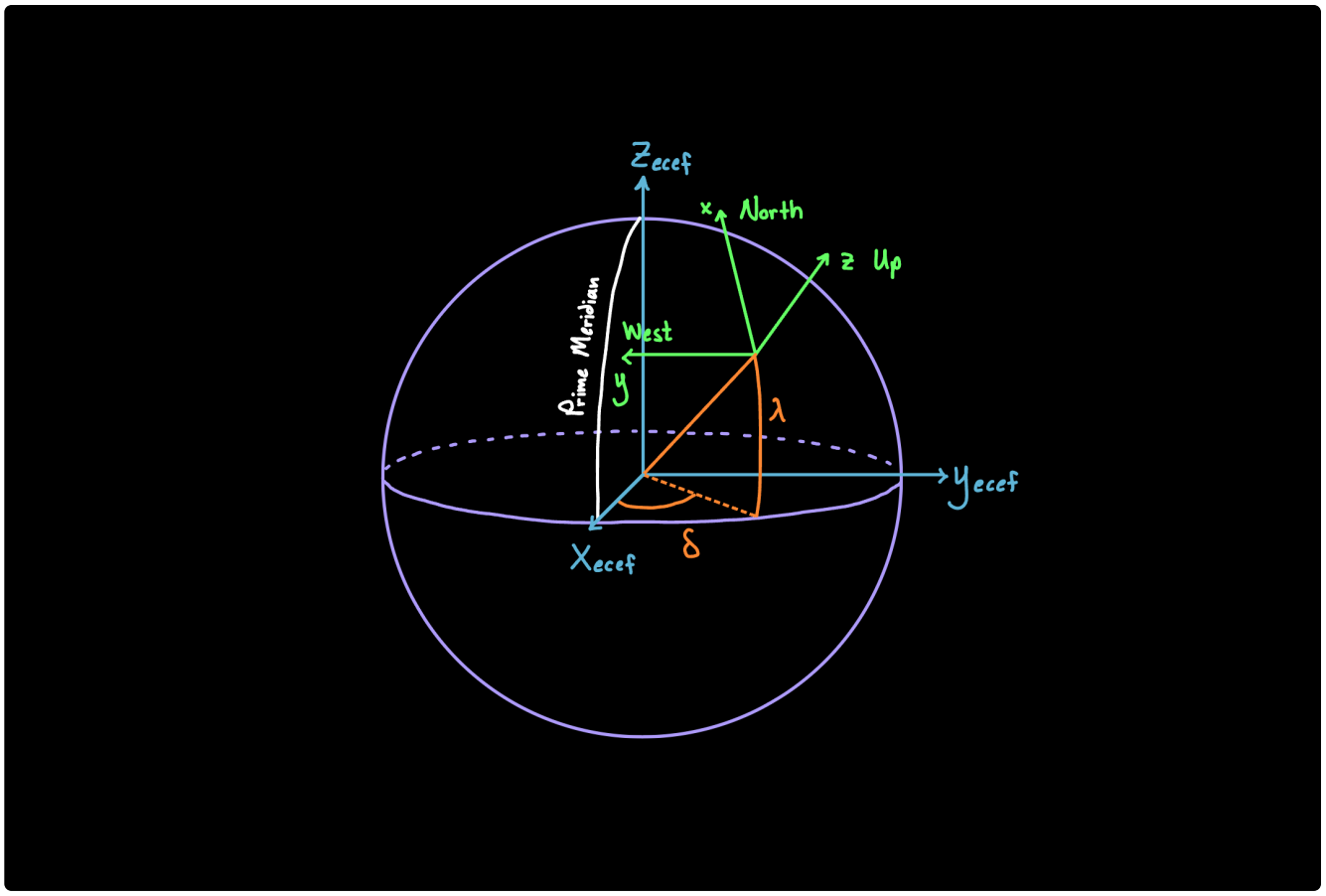
More importantly, they understand **why** the robot reacts the way it does. They know how input shaping affects traction, how current limiting changes acceleration behavior, how PID or rate limiting influences stability, and what performance trade-offs come with each adjustment. This transforms practice into an iterative optimization cycle: test → analyze → modify → validate.

In a competition environment where milliseconds and small corrections matter, this technical awareness reduces unpredictability. The robot does not just become "comfortable to drive"; it becomes intentionally tuned for reliability and repeatability under match pressure.

Related Articles:
https://community.firstinspires.org/hubfs/web/program/frc/resources/improving-driver-performance.pdf
https://www.chiefdelphi.com/t/cycling-optimization/342304
https://info.firstinspires.org/hubfs/web/program/frc/resources/selecting-drivers.pdf
https://team2168.org/images/stories/OrganizationDocuments/Driver%20Training_8.22.11.pdf

Terminal Dashboard

Terminal Dashboard idea was created because of the need to tune the coefficient values for the driver. it started as a calculator for our logaritmic drive code. We can see the changing of the motor-output according to the coefficient values. Then we added joystick implementation for clear **understanding** and added a controller outputting mode for diagnosticing the controller and seeing the values even we are not with the robot. Then we wrote a module for calculating the shoots and digital-twin algorithm. (writing this module gave us a clear sense for calculations, and helped to initalize the code) This module creates a analytic plane and position the game elements(HUB, TRENCH, BUMP, APRILTAGs) according to 2026-field-dimension and WPIlib doc.
Note: the field lenght is 16.540988 meter but we take it as 16.54
The cordinat system use NWU (North-West-Up) system according to WPILIB.
NWU(North-West-Up)
"This system has the x-axis pointing to the North, the y-axis pointing to the West, and the z-axis pointing away from the surface of the Earth."



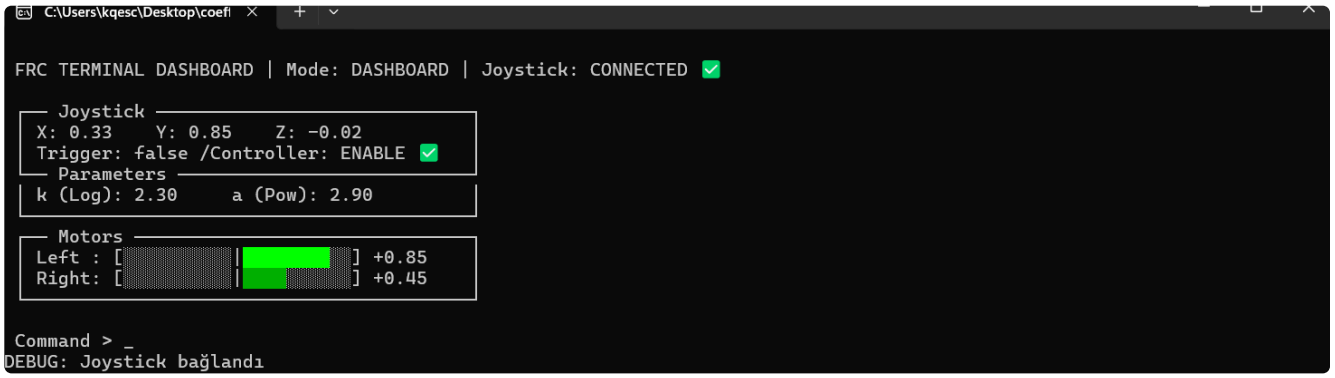
This is why it feels a little weirder than a regular analytic plot. The *dots* analytic locations writed below.
$P^b_{left}(0, 8.21)$
$P^b_{right}(0, 0)$
$P^r_{left}(16.54, 0)$
$P^r_{right}(16.54, 8.21)$
$P^b_{hub}(4.57, 4.10, 1.12)$
$P^r_{hub}(11.97, 4.10, 1.12)$
$P_{id:1}(x_1, y_1, _1)$
$P_{id:a}(x_a, y_a, _a)$
$f(x) = 16.542$ (middle line)
Why we use TUI? Actually using GUI with html/css was an easy and fast solution. Also i was used SDL2 library for getting joystick input and it is capable of 2d graphical rendering (so we can make a GUI with it). Using TUI was an personal aestetic preference.
We use **bubletea** library

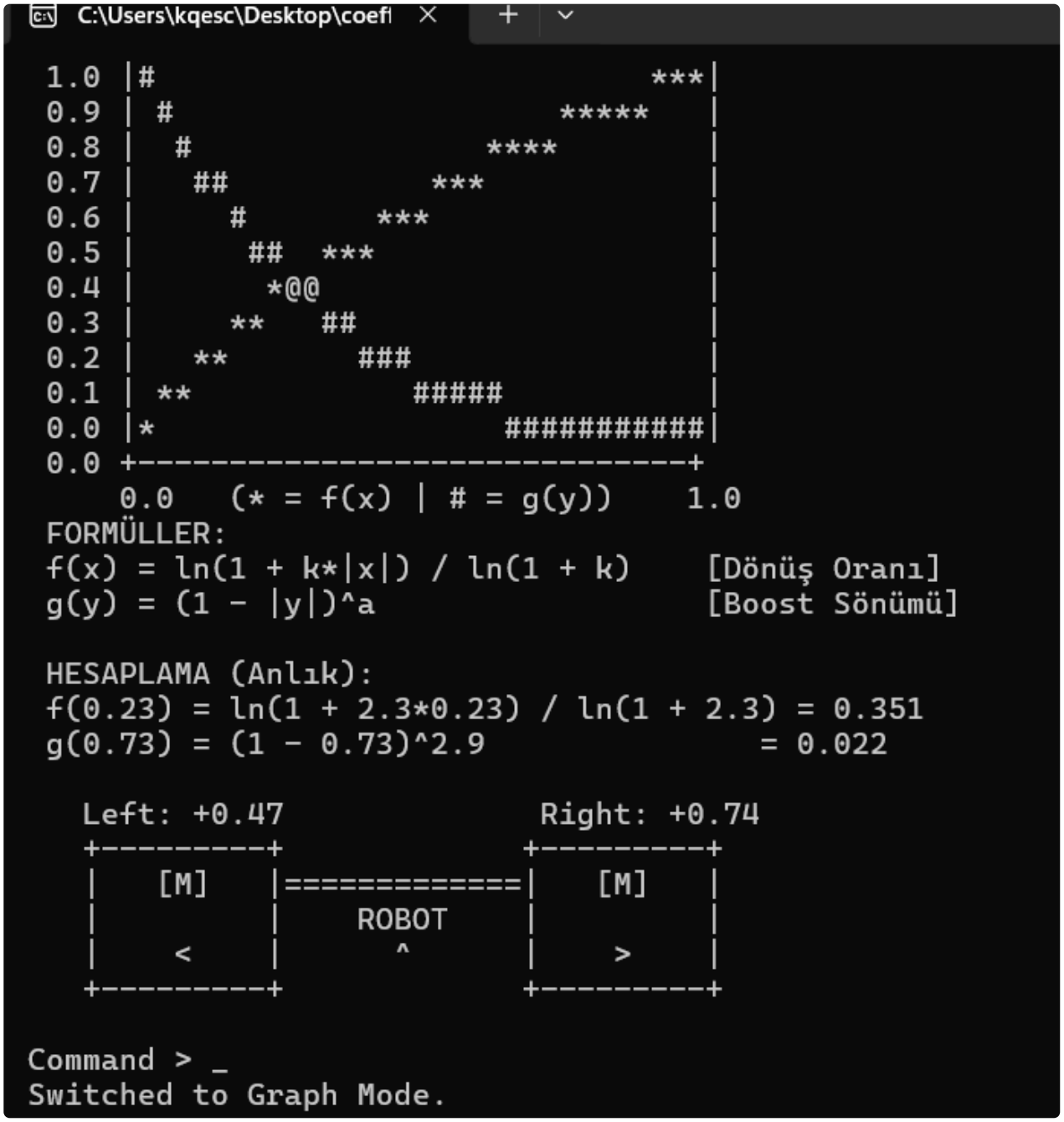
https://github.com/veandco/go-sdl2

https://github.com/charmbracelet/bubbletea

https://iansguides.com/tutorials/coordinate-systems/

https://github.wpilib.org/allwpilib/docs/release/java/edu/wpi/first/math/geometry/CoordinateSystem.html





https://github.com/TeknoNFL-8242/Terminal-Dashboard

RPI / Cam / Photonvision

Apriltag usage is common and community supported phenomenon. It is a simple and effective way to complete the autonom period especially. You also just need a camera (may not work with old cameras). Usage of Apriltags makes difference for who don't have much experience in developing autonomous. Moreover it is also essential for the experienced teams too.

As a team which already has some experience in autonomous (PhotonVisuon too), I am gonna talk about our experiences about PhotonVisuon and RPI.

For Apriltag perception, we were installed PhotonVisuon image onto RPI5. We chose OV9281-160. The cam is not compatible directly with RPI5, we bought a adapter(MIPI). We were imaged the RPI5 with latest stable PhotonVisuon release (v2026.2.2). At the beginning we suppose that it works fine but the camera did not recognised on GUI. We think that the problem was software-releated and connect to the PhotonVisuon image throught SSH. With "dmesg" command, we realise that OV9281(the cam) is connected. After that, we tried "v4l2-ctl" command but the camera is not detected. We were considering that it could be about outdated-packages but updating packages did not work. Then we searched "boot/firmware/config.txt" file and customized that. We changed the configuration and restarted over and over again with all standard and non-standard camera parameters but didn't work. We reinstalled and changed the PhotonVisuon image again but that was not give us a solution. Then we realised all the time we overlooked the possibility of "hardware-related" error. My teammate looked at the camera again but he did not assumpt anything this time. He realise that there was a 3.3v pin and a gnd pin.
https://docs.photonvision.org/en/latest/docs/quick-start/index.html
https://github.com/PhotonVision/photonvision/releases

Lesson Learned
Don't forget to look at the documentation(even if you think that you know how to use the electronic component). Rewieving official manuals could give you a clear understanding of how it works. Also sometimes the wrong assumption that you already know something (it does not matter how simple it is) could mislead you. Verifying the assumptions briefly could save a lot of time. Spesificly this incident did not appear because of overlooking the manuals but looking at the manuals prevents you from making these kinds of attention/assumption mistakes.

Two Weeks Before The Competition

The following workflow writes below was completed in the last two weeks leading up to the competition.
Suggestion: Actually it was really challenging, limited time and so much thing to do. Both for software and mechanic, even for PR.
Yes limited time is compelling but if you go systematicly, work hard and manage your time effectively, it becomes managable.

Tactical Analysis (Yes, again)

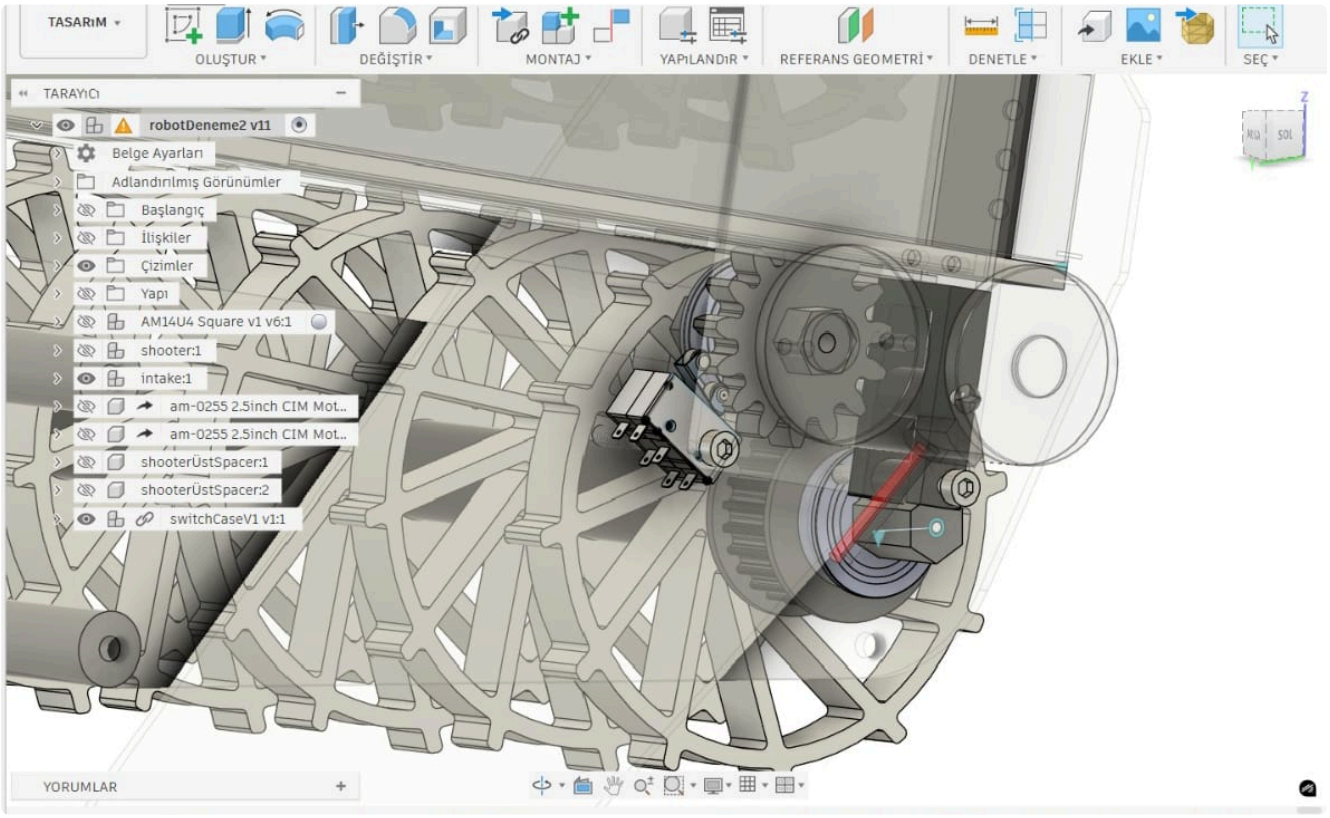
As a software developer (closely related with autonomous and teleops autonomation) and also a driver, looking for FRC match specific tactical resources and thinking alternative strategys of course increases the autonomous and the drivers performance. Don't forget, the true strategy can return as win and it even works against quite prestigious opponent alliances.

Mechanic

Intake:

Our intake mechanism extend with a rail mechanism. For a better (automated) drive feeling we use 2 micro-switches (that was the plan but we changed this a little bit). Switches gives status of the intake mechanism (opened fully, closed fully, somewhere in the middle). This implementation also secures

the extend mechanisms relaiblty. (Not: We notice that it could be suitable for the system in january, but it took us until the last week to finalize it) We ordered different switches to decide which one is suitable (the switch list: dc162, dc163, dc166, dc 168, dc170, dc171)
In 2025 REEFSCAPE season, we built a elevator mechanism, the microswitches is suitable for the elevator mechanism too. It gives the status of the elevator and secures the elevator from arbitrary forcing with a easy software implementation.(The elevator could at his top but if you keep the motors running, it may broke. Adding this switches is simple security system.) Also in 2025, you could attach the corals to reef. The reef has three height level called L1,L2,L3. You can also use this switches for stopping the spesific lenght for coral attach. (You can use hall effect sensor too, but be carefull with magnetic field and other factors. Micro-switches are more stable)
You can customize this for your use case. I give this for gaining you a perspective.
The switch which return "is the intake status closed" can positioned fine but the swich which represents the "fully opened" status couldn't positioned as planed because of mechanical difficulties.
Alternative Solutions:
The switch can open with a ropes tensions but it is dangerous (especially thinking about the FRC matches wilderness)
The "fully opened" status could give with a TCRT5000 IR sensor. This kind of sensors are using for making a "Line follower robots" competitions.
"It can also be used to check the colour of something on a black to white scale. This is a principle a line following robot can utilise. The different shades change the level of reflected infrared light."
https://www.instructables.com/TCRT5000-Infrared-Reflective-Sensor-How-It-Works-a/
Painting the alimunium intake profile(dark gray colored) with white probably solves the problem.
Update: We found a way to impementing open
A problem can give you a solution:
We thought that we can use this swiches as encoder. Those swichs have a roller at its end. When the gear has moved, the swich open and close. (also it is not skip gearteeth if it is placed by adjusting).
The elevator motor is working low speed so no much corrosion. But we didn't use it because our intake system is unreliable ,automating may cause some brokes, it didn't open-close as expected.



Shooter (Flywheel):

A flywheel loses speed when it contacts the ball. The RPM (Rotation Per Second) decreases and RPM directly affects velocity . To compensate the velocity drop, we used an algorithm named [PIDF] (https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller. We will talk about this in later section.
To use this algorithm, an encoder is required. So we added an encoder to our shooter mechanism shaft. We preferred the"Rotary Encoder E6B2-CWZ6C 200 Pulse"(Rotary Incremental Optical LOW–MEDIUM resolution(200 PPR)). PPR means Pulses Per Revolution. That means 1 rotation (360') equal to 200 pulse. That means each pulse equals to 1.8 degree of rotatin (360/200=1.8). A/B channel (quadrature) feature is increases the sensitivity much more.

We initially selected the encoder's 600 pulse version because we assumed that higher pulse count means higher resolution (prices are similar). Yes it is true but higher resolution means higher microprocessor load. For a shooter, we care about **velocity**, not absolute position.
Also buying an encoder saved us a ton of analysis work. I will talk about "what was that?" in later sections.

https://en.wikipedia.org/wiki/Proportional%E2%80%93integral%E2%80%93derivative_controller
https://www.chiefdelphi.com/t/quadrature-encoder-channels-a-and-b/416570

https://www.akm.com/global/en/products/rotation-angle-sensor/tutorial/type-mechanism-1/

https://www.rls.si/eng/encoder-handbook/how-to-select-an-encoder?gad_source=1&gad_campaignid=9031401113&gclid=CjwKCAiAkvDMBhBMEiwAnUA9BS9xuJjQ11xJk66aEOzMxkZu9ip1sEaWYyEcxbkW4B9cSKfPe8ikPxoCHW0QAvD_BwE

https://www.ia.omron.com/data_pdf/cat/e6b2-c_ds_e_6_3_csm491.pdf

https://www.rls.si/eng/encoder-handbook/how-to-select-an-encoder?gad_source=1&gad_campaignid=9031401113&gclid=CjwKCAiAkvDMBhBMEiwAnUA9BS9xuJjQ11xJk66aEOzMxkZu9ip1sEaWYyEcxbkW4B9cSKfPe8ikPxoCHW0QAvD_BwE

Not: Self-manufactured A-3144 (hall effect sensor) based magnetic encoder (Explained at "Odometary" section) could adapte to the shooter too, But this solution was experimental. Also the consistenty of Self-manufactured appeared because of the financial problems. We were planning as analysis based **trial and error** solution. This process has been explained in later sections.

Manual controll (For subsystems):

Our goal is shooting without any driver intervention. But for some situations we wrote the functions that controls the subsystems(intake mechanism{includes: intake elevator, intake motors}, Flywheel shotter{includes: ball lever, flywheel motor})
Added diagnostic code for the mechanical and added this below the Test mode. This is also helping to the mechanic team for testing their structures statu/performance.

In fact: It didn't help the mechanical team as much as it normally would have. Because All of the development process handled in last two weeks. No time to analysing status/performance. If we had a mechanical problem, we tried to make it work as much as possible because of the time limitations. The mechanical team tried to manufacture as reliable as possible so we focused on essensial test and speed up the analysis phase.

We see the importance of a good time management plan again. Event though it worked, it could have worked better with a good analysis and customizations.
Writing a Test Mode is also great for verifying our functions status, such as autonomous routines, AprilTag perception, PID, PIDF, and more.

Electronic/Hardware & Software

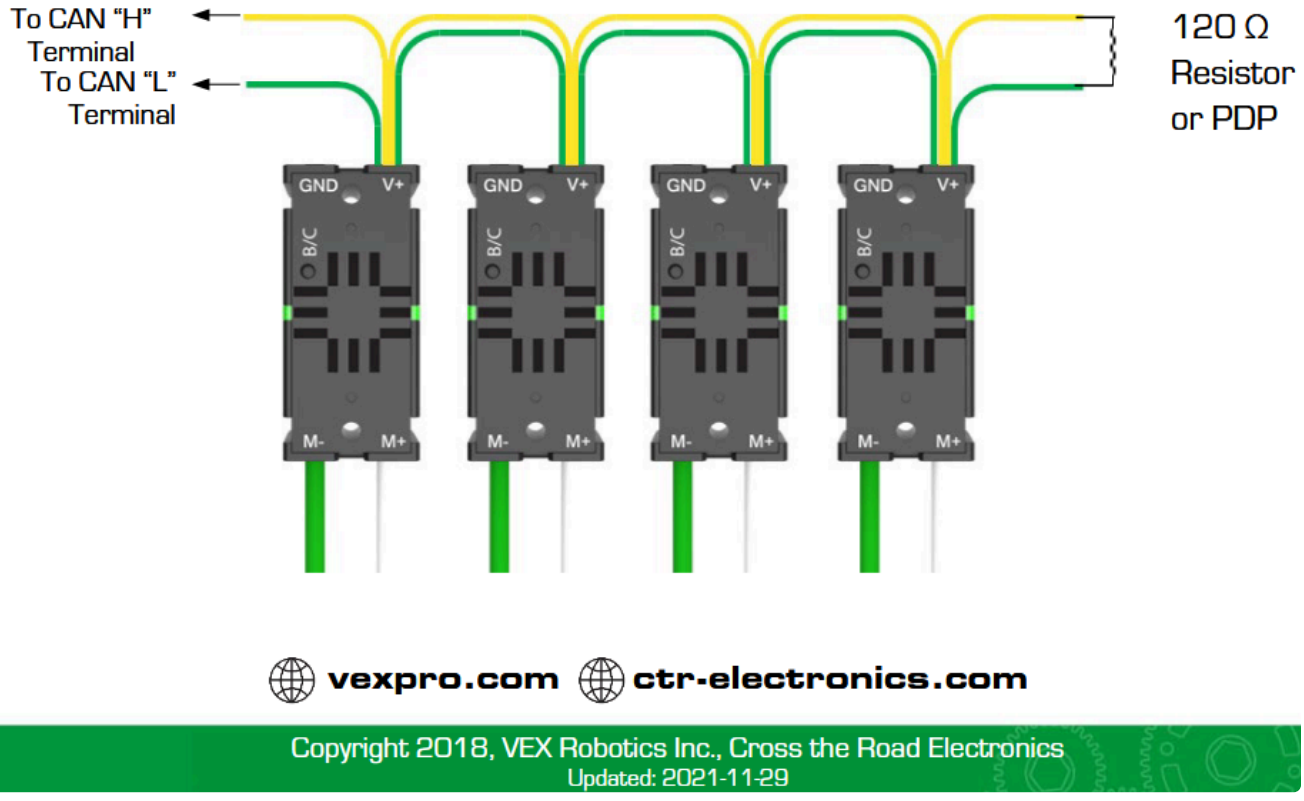
Electronic-Side:

What is CanBus

The Controller Area Network bus is a kind of a network protocol/communication system that provides communication with industrial devices. Via canbus, the devices which is connecting to network can exchange data
https://www.cartrack.co.za/blog/what-is-canbus-and-how-does-it-work

Canbus relies on two cable, CAN High - CAN Low; Also in CANBus, there is no central controller like router.

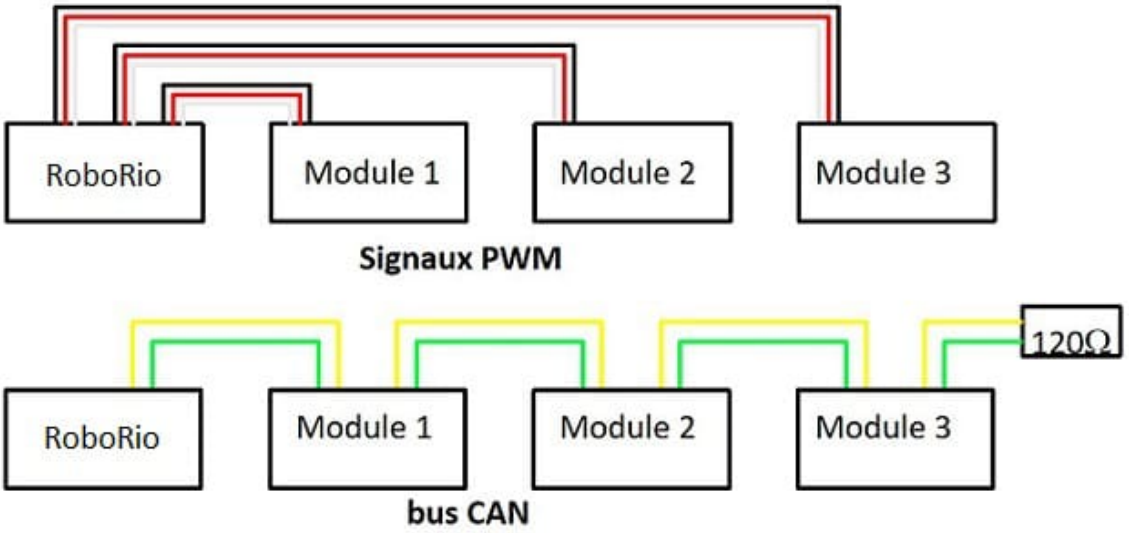
Between two cables, we should add 2 120 ohm resistor for CANBus termination But for FRC. But FRC's PDPs have its BUSTerminator and also RoboRIO had (Terminal Device). So normally you don't really need to add resistors to your CANBus network for FRC usage.



https://sitaltech.com/can-bus-termination-testing-a-step-by-step-guide/
https://www.csselectronics.com/pages/can-bus-simple-intro-tutorial
https://docs.wpilib.org/en/stable/docs/software/can-devices/can-addressing.html

For FRC CAN devices Specifications: https://docs.wpilib.org/en/stable/docs/software/can-devices/can-addressing.html#frc-can-device-specifications

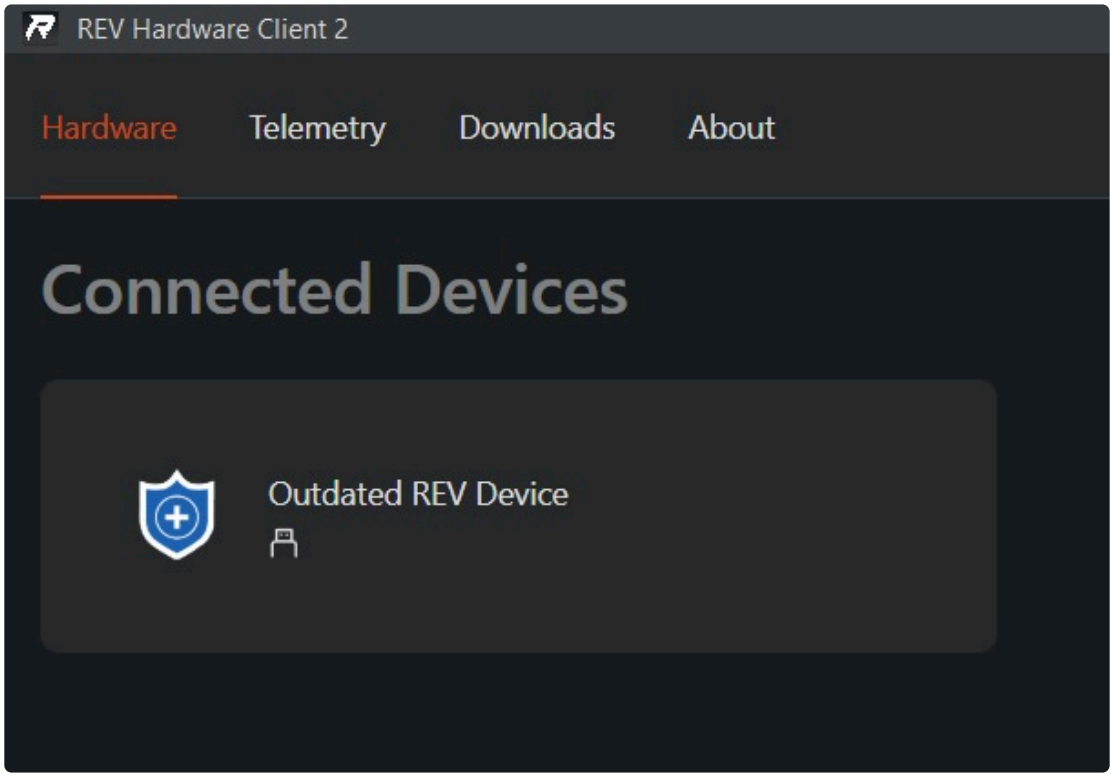
CANbus & PWM



CanBus Fail & SparkMax & VictorSPX

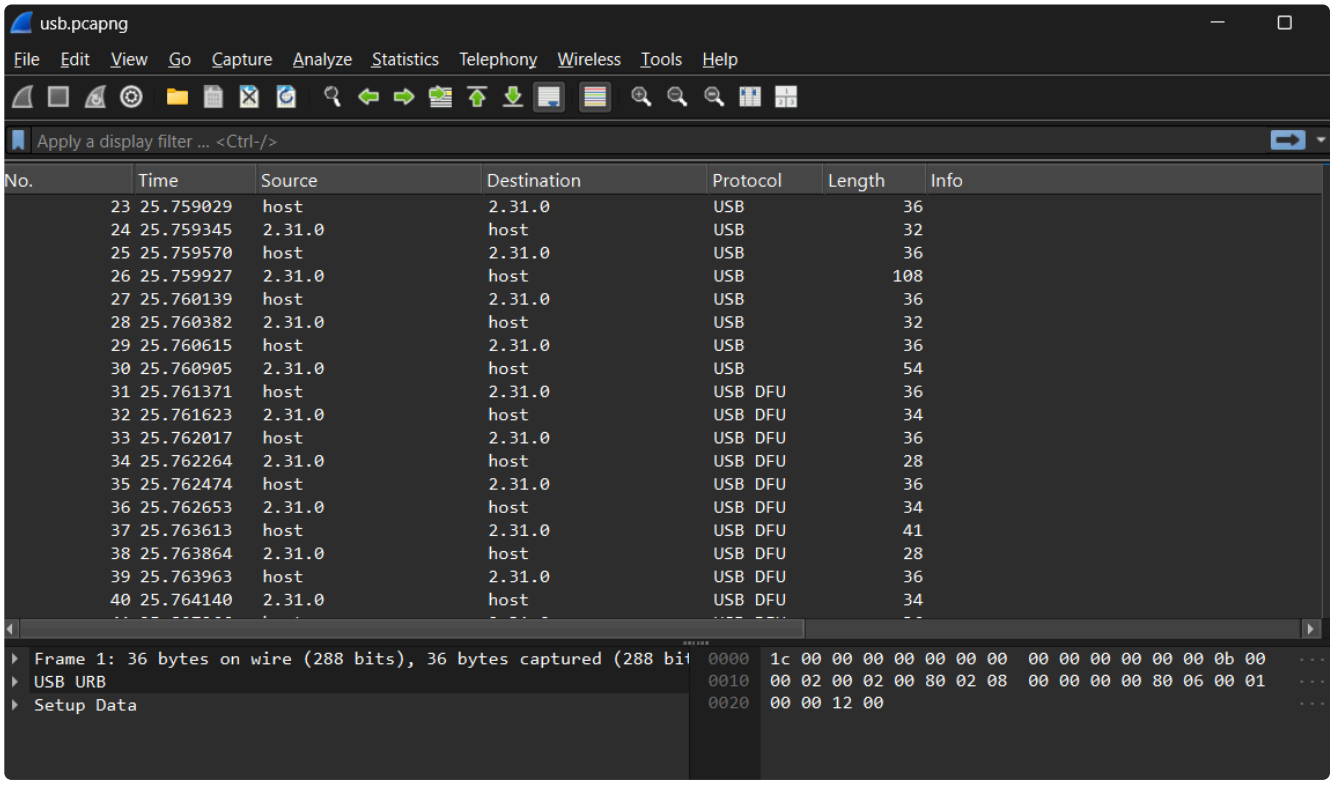
We were using 4 VictorSPX at the begining. They were operating normally. But we add 5 Spark Max to our CANBus network. We connected all of them but not worked. Spark Maxs didn't work as expected, also the VictorSPX -before Spark Maxs connection, it works okay- were Blinking Red (Status: Failed Calibration)
https://ctre.download/files/user-manual/Victor%20SPX%20User%27s%20Guide.pdf

You can configure REV Robotic products with REV Client . One of our computers sees all the devices as Outdated. But when we change the computer is sees as expected. We made firmware update and it works succesfully but even after that. When the Spark Maxs connected to other computer, it still seen as Outdated device.



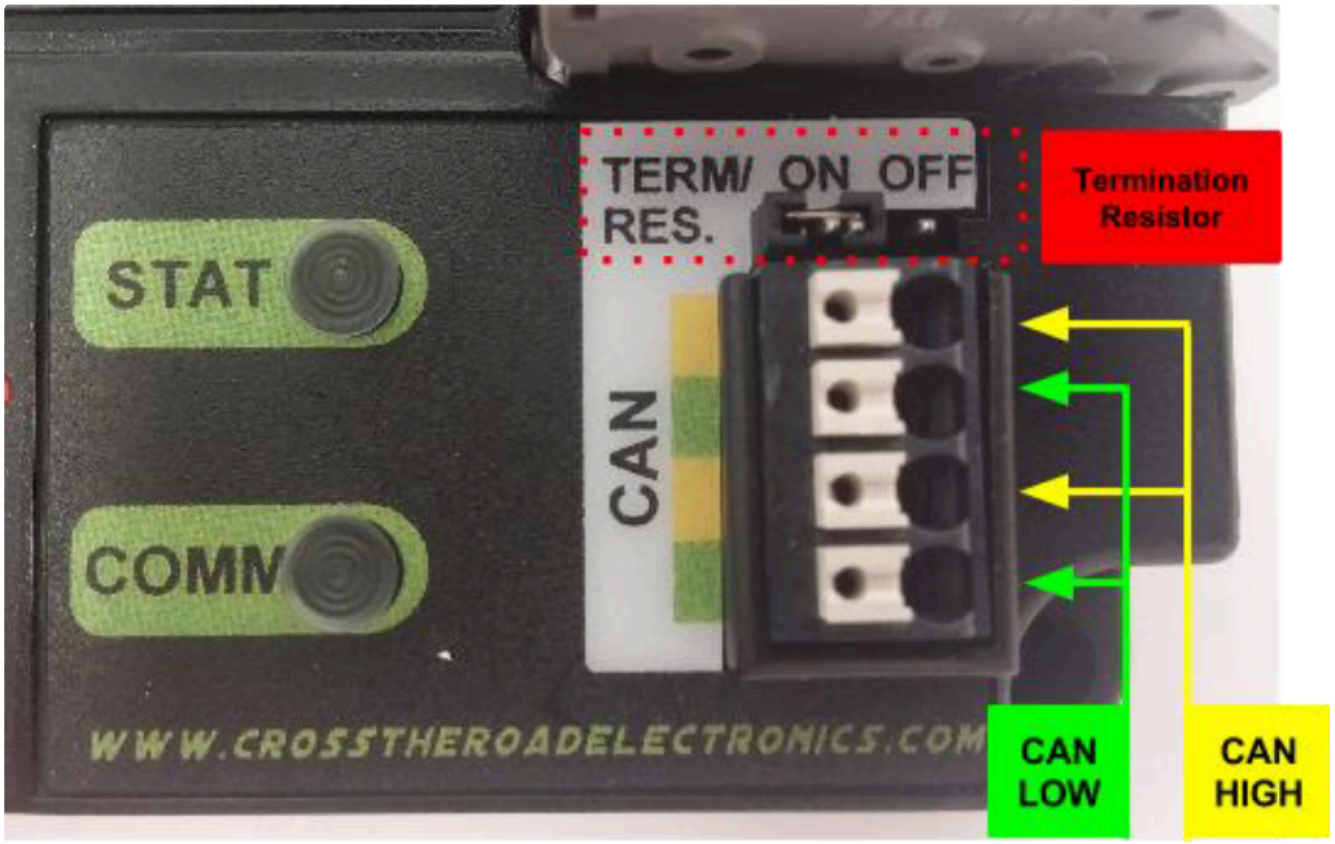
Spark Max Status The lights indicated a problem.
One of the Spark Max lights was flashing red: Pressing the Mode button to enter recovery mode worked, but it gave an error again when the flashing stopped. We tried the flashing repeatedly, contacted the driver, but unfortunately, the problem was not with the driver. To solve this issue, we tried all the methods we knew, such as hard resetting and hard formatting directly to the data gateway. Then

we returned to software solutions, such as Zadig software and packet tracing, writing data using a kind of bridge by changing some protocols. These didn't work, so we realized we had to manually write the packets and data directly to the chipset. However, we had no knowledge or experience in this area. And so we decided that the Spark Max was bricked.
https://www.chiefdelphi.com/t/is-there-a-way-to-unbrick-a-spark-max/343866
Also i intercapt the traffic with USBcap mode Wireshark. I save the recovery traffic and save for later debugging.
https://docs.revrobotics.com/brushless/legacy/spark-max-client/recovery



Also another Spark Max didn't even seen by any of our computers. We thought that Type-C socket broked. But when you put the Type-C cable some spesific angles, Spark max has seen from the computer and we flashed it.
https://www.chiefdelphi.com/t/is-there-a-way-to-unbrick-a-spark-max/343866

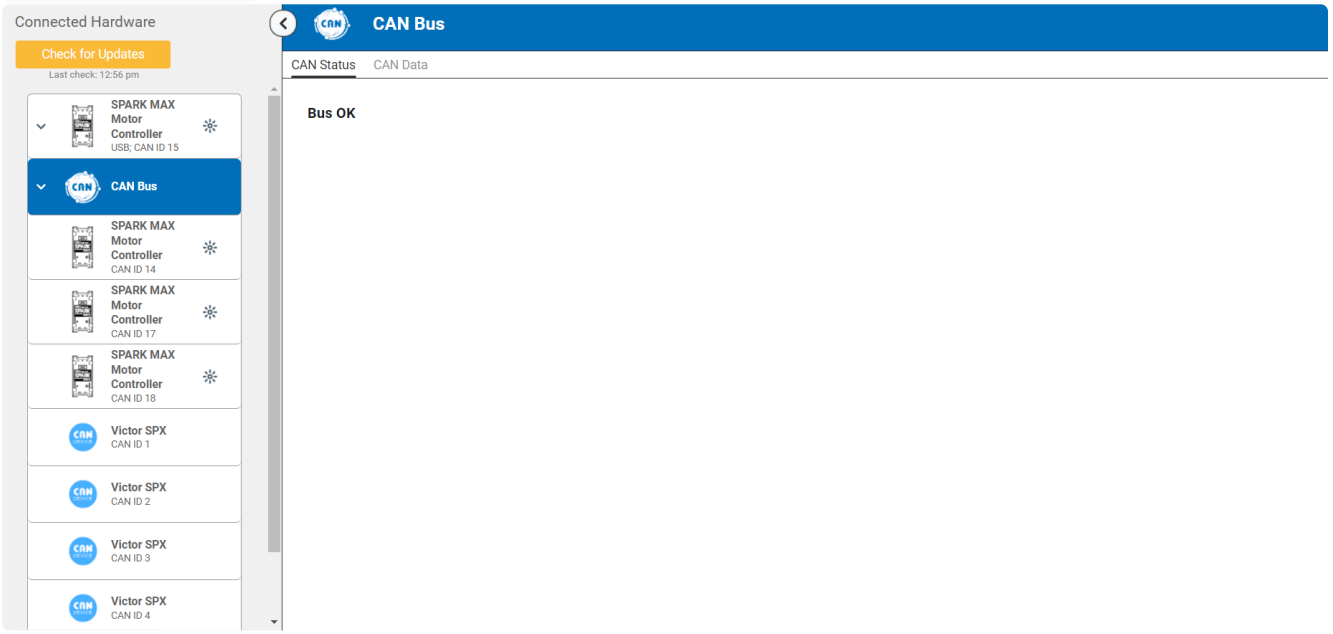
But the issue is not about the devices all time. When we use phonex tunner, all the electronic speed controller devices seen, also we looked at Rev Client and all the devices have been seen but exclude PDP. Also VictorSPX devices returned red blink as i said earlier.
https://v6.docs.ctr-electronics.com/en/stable/docs/tuner/index.html



https://ctre.download/files/user-manual/PDP%20User%27s%20Guide.pdf

I looked at PDP(CTRE) documentation. There is a termination button we changed this but didn't work. We openned the PDP but no physical damage detected. The Status lights RED(no Blink) and there was no statuslight describes "Static Red light" in documentation. We unplug CAN cables from PDP. The VictorSPX status lights blinking red at the beginning but after 20-30 seconds, status color change to "Orange blink" (CAN Bus detected, robot disabled). It operates okay (İt worked but CANbus status - from REV Client-) but when you connect CAN-H and CAN-L cables it fails. We disable the Termination resistor in PDP, connected the CANbus connection back and connect 120 ohm resistor to other CANbus socket. It didn't work too. Then we unplug CAN cables from PDP and directly solder to 120 ohm resistor. The VictorSPX status light habit didn't change but when i check the CANbus connection from REV Client

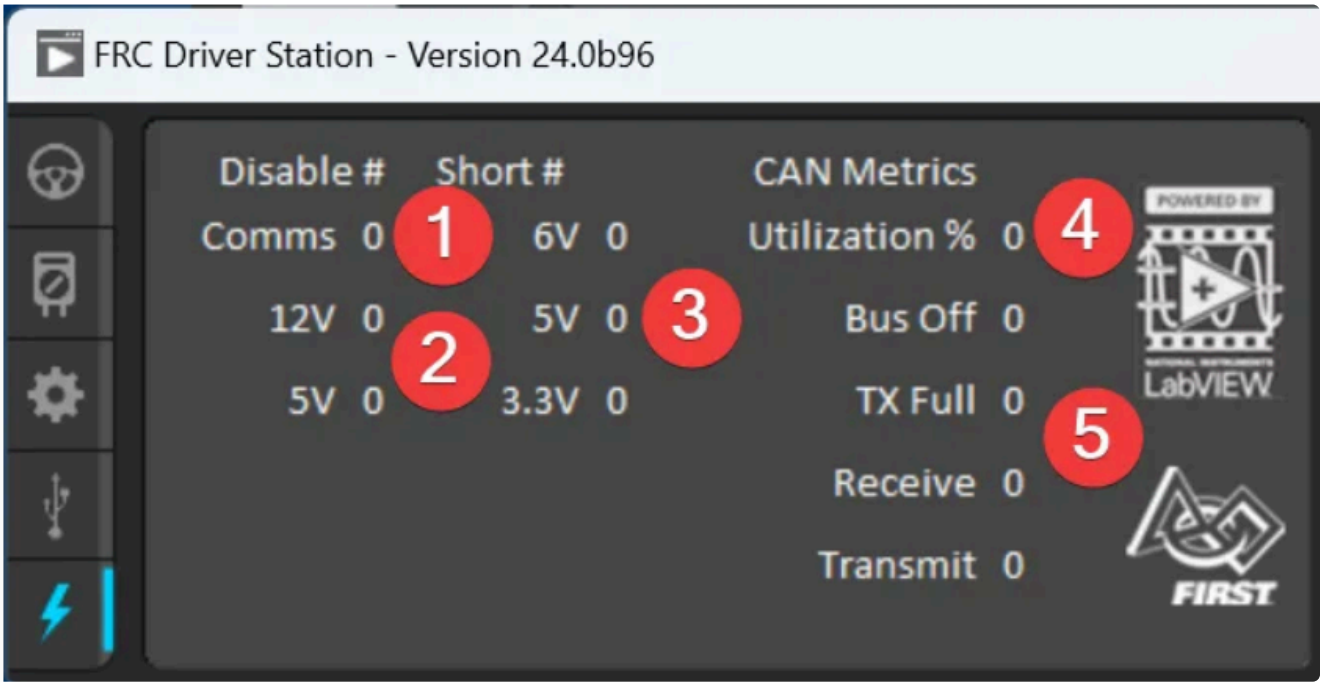
`"Bus Status: OK"`



the warning messages didn't appear. Looks like we solve the problem. But i supposed that the voltage

value was getting from PDP wia CANbus but we still can get the voltage value from the Drive Station. After the season we will try to repair or change the PDP.

NOT: You can check FRC DriveStation to see CANbus status.
https://docs.wpilib.org/en/latest/docs/software/driverstation/driver-station.html#can-power-tab



*__!Lesson learned:__
One day, i tried to control a motor with RoboRIO(java). I check the CANbus ID, check the code nothing seems anormal, i looked at the sparks CANbus cable, the problem appeared because i unplug the CANbus cable for configuration, motor testing, i forgot to plug it again. This carelesness cost me about one hour.

DIGITAL-TWIN

Hardware-Side:

Camera (for Apriltag):

We bought OV9281-160 cam (monochrome) camera for apriltag perception. We preferred a monochrome camera because it is cheapper than a colored camera (compared to its alternatives with similar performance but had color) and "monochrome camera sensors are capable of higher detail and sensitivity than would otherwise be possible with color"
https://www.red.com/red-101/color-monochrome-camera-sensors
Also we had a ps3 camera and some cheap webcams as backup.
!Lesson learned:
We tried to connect the ps3 camera to the roborio directly in 2025. It did not work (other usb cameras is working well but ps3 camera need a driver, it is not a standart UVC cam). We tried some interesting technicques like connecting to roborio with ssh and installing the ps3 driver. None of the solutions that we tried did not work.
All of the time, the solution was just in front of us. We can use RassberryPi5 for using ps3 cam. PhotonVisuon image supports -Debian based Linux systems includes gspca_ov534 (driver)- . It is better and simpler solution (check the cam status and look at the documentations when you are struggle).
This mistake forced us to play the matches with a cheaper camera (model: HWK-W WEB).
before a week to the competition, we decided to buy a second RPI cam. Actually we decided this because RPI has 2 cam socket (2 x 4 lane MIPI DSI/CSI connectors and we got some money. We buy a monochrome camera again (OV9281-110).For our autonomous routines, color-based object detection was not necessary. Precise localization was. so we made a trade-off with color perception(with this, we can see the fules forexample) to faster and reliable visuon processing (increasing apriltag percaption performance).

But what was the requirement. Actually theory is simple. I am a human as a part of Bilateri subset (One defining feature of Bilaterians is bilateral symmetry — including having two forward-facing eyes.). Being with two eye gives "Depth Perception".
Monocular (One-Eye/camera) Vision
With one camere/eye (image output), : True stereoscopic depth perception (stereopsis) is not avalible in Monocular visuon. Monocular Visuon deept perception relies weaker clues: Perspective (parallel lines converging), Relative size (smaller objects appear farther), Motion parallax (near objects move faster across the field of view), Shadows and lighting, Occlusion (one object blocking another) (For living things but same principles could implement to a camera too.)
Binocular (Two-Eye/camera) Vision: **Describing the visualization (same scene) process with two-eye/camera. This provides the** real stereopsis**. This is because
- A nearby object appears at different horizontal positions in each eye.
- A far object appears almost at the same position in both eyes.
-Triangulation
So being with two cam may help for taking more reliable distance output (of course in theory).
We thought to adding two camera in front of the robot for Bincular vision. But WPILIB does not have dirrect support for stereopsis depth perception. Also if we tried to code, this type of visuonisation is a complex process and we have limited time.
https://eyewiki.org/Monocular_Precautions

How PhotonVisuon Calculates the Distance

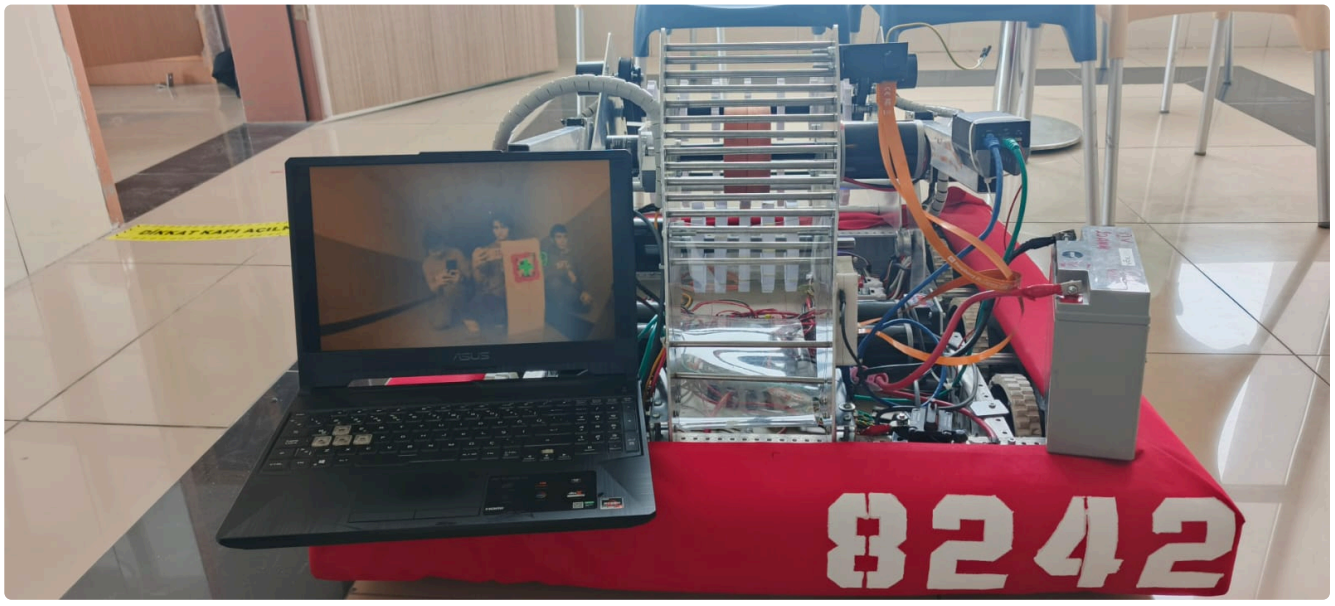
I said Monocular visuon relies weaker clues about the depth (distance) but PhotonVisuon can calculates almost exact distance between camera and apriltag. How does this process happens? PhotonVision uses OpenCV's **solvePnP()** algorithm to compute the tag's 3D pose relative to the

camera(3D pose estimation). The apriltags sizes is predefined, also the exact 3D coordinates of their corners are known.

https://github.com/wpilibsuite/frc-docs/blob/main/source/docs/software/vision-processing/apriltag/apriltag-intro.rst
https://github.wpilib.org/allwpilib/docs/2027/java/edu/wpi/first/apriltag/AprilTagDetection.html
https://docs.photonvision.org/en/latest/docs/programming/photonlib/using-target-data.html
https://docs.photonvision.org/en/v2025.3.1-rc1/docs/apriltag-pipelines/coordinate-systems.html

We plug two cam to RPI5 camera module port and calibrated with ChArUco Board. We tested the distance calculation method, ±20 cm (generally calculate less than the real distance) tolerance. It may be because it hasn't been properly calibrated. One more calibration could solve the problem.
We decided to locating the cameras opposite direction as one camer looks forward and one camera looks backward. Pose estimation by multi-cameras are supported by WPILIB. If the camera which is located forward couldn't see any apriltag, if backward-cam sees a apriltag we still know the robots position.

https://www.chiefdelphi.com/t/multi-camera-setup-and-photonvisions-pose-estimator-seeking-advice/431154
https://www.chiefdelphi.com/t/best-practices-for-running-multiple-cameras-photonvision-and-orange-pis/507551



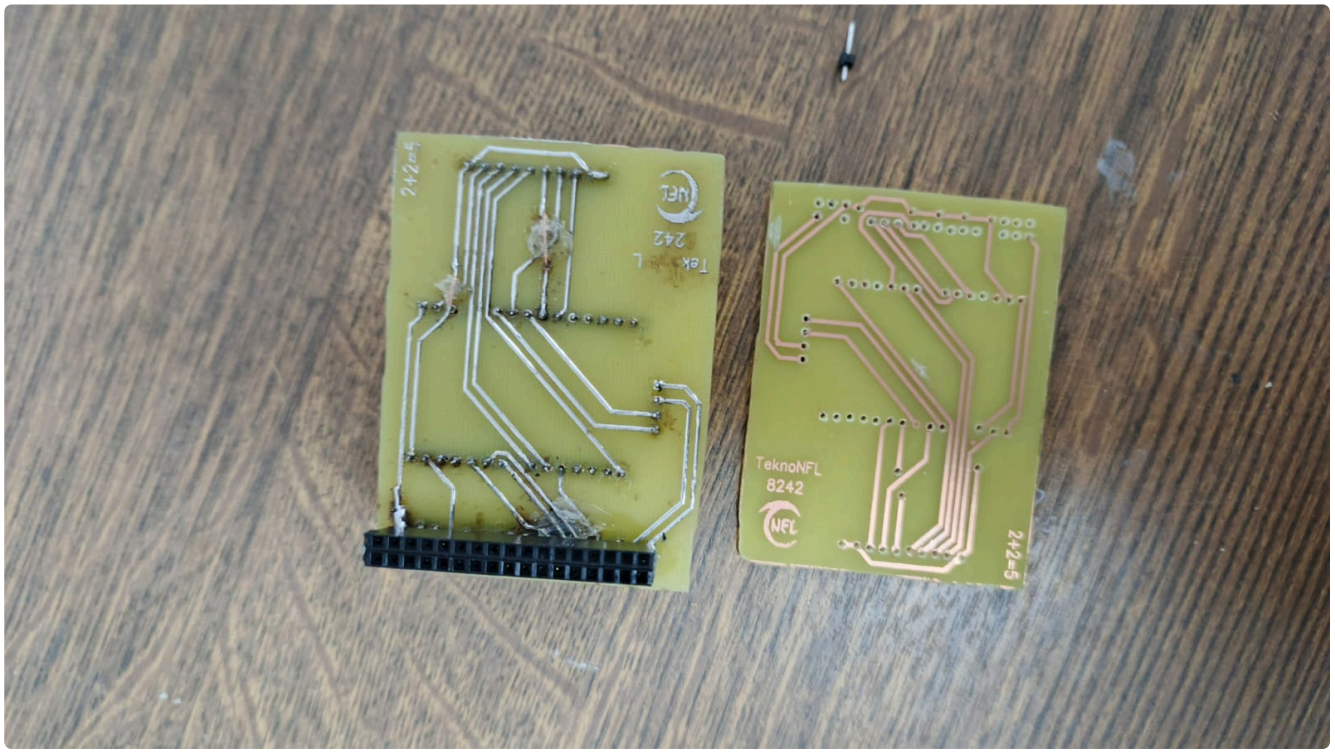
Actually, the Digital-Twin system is not just about AprilTag pose estimation. We create diffenet integarion algirthms for handling diffenet imputs. However, since the deadline for the first release of this article (İstanbul Regional, 2–5 March) is very close, I decided to write this section in a later release.

GYRO-IMU

Why we need gyro?
"I would like to thank Muhammed Salih TÜMKAYA for his dirrect contributions to this section."
Gyro usage is necessary for our Digital-Twin system, the autonomous period, the automated shooting algorithm we will use in teleop, and PID (I will talk about these in more detail in later sections). Because of that, we needed a gyro sensor with high stability. First, we looked at the gyros that are officially supported in FRC, but they were being sold at a high price. Because of that, we turned to the ICG 20660L gyro sensor. But due to the customs regulation that came on February 6, we looked at

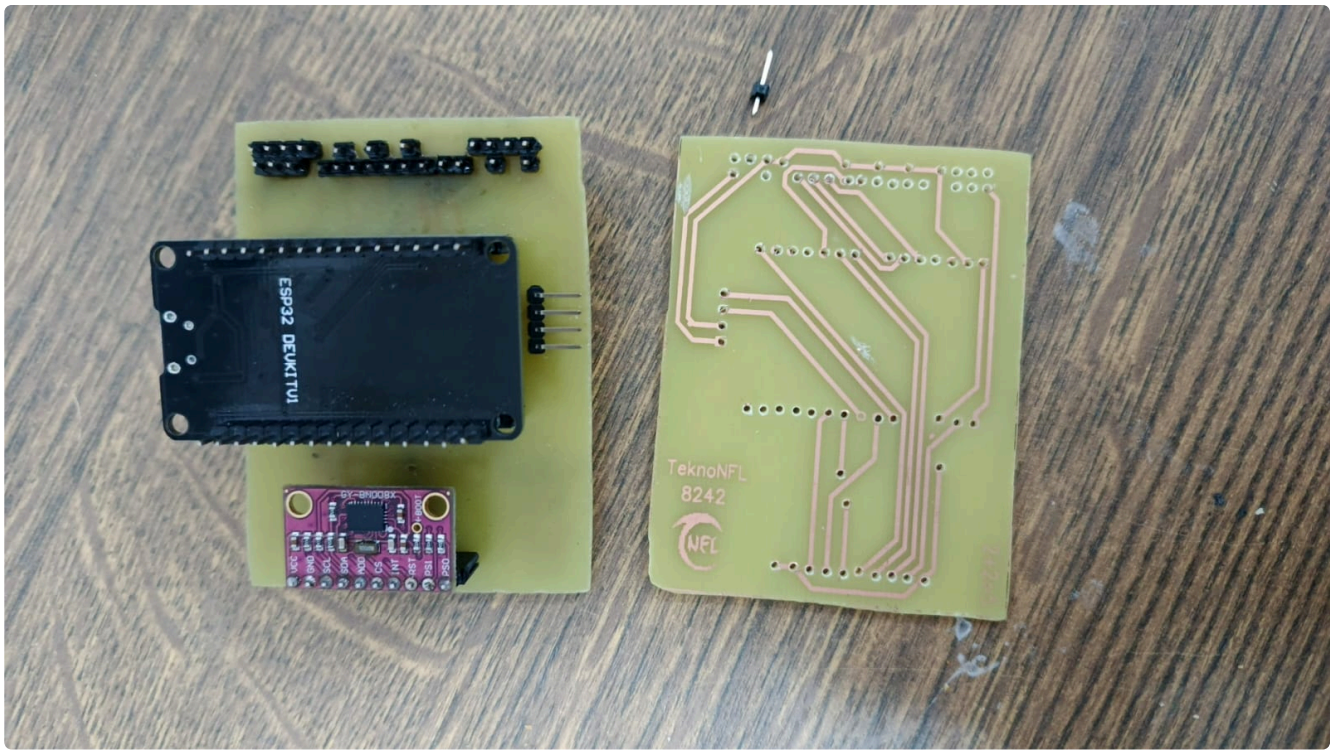
products that are distributed domestically. We selected the MPU9250 sensor because it was price/performance-friendly and stable, and it also had a magnetometer. We ordered three of these sensors.

By using three sensors at the same time, we aimed to increase accuracy with some algorithms we planned and used, and to identify the gyro that outputs wrong values to apply a kind of filtering. But since the gyros we bought turned out to be fake, we didn't have the hardware capability to implement this mechanism. Because of that, we used the MPU6050 gyro sensor that we already had. This sensor is low-budget and not a stable gyro sensor. Still, to reduce the error rate down to the theoretical minimum, we used filtering and method approaches (Madgwick, Z-Stop, Kalman, DMP). https://qsense-motion.com/qsense-imu-motion-sensor/madgwick-filter-sensor-fusion/



Later, close to the competition, we found additional budget that could be spent on robot building. With this budget, we decided to buy the GY-BNO080 / BNO085 gyro sensor, because it provides more stable results since it performs the filtering process inside its own processor.

After that, we found this library written for that sensor using ESP32-IDF: https://github.com/myles-parfeniuk/esp32_BNO08x. And to imitate professional gyros with our own resources, we designed a custom PCB for the MXP port. Basically, we designed ESP32 as a middle layer between the gyro and the RoboRIO, and made a design that transfers the gyro data from the ESP32 to the RoboRIO via SPI over MXP. We produced our PCB on a single-sided FR1 copper board using toner transfer paper, and etched it with ferric chloride to dissolve the copper areas except the toner. Then we soldered the pin headers, and to imitate a double-sided structure we soldered copper wire over silicone. And with our own financial resources, we managed to build a mid-stability gyro board that uses the MXP port.



Odometary

The traversal information is a importand input for our field-navigation-system (Digital-twin). If we know the traversal distance, we can update the location according the last known location and GYRO information (last known location information came from Apriltag Pose estimation functions)
So with odometary, we still know where we are even there is no sudden Apriltag information.
We search for different odometary models.
There is a dead whell odometary method but we didn't prefere that. Knowing the drive trains whell traversal is more reliable way. So because of that we looked at encoder models.
The "Through Bore Encoder V1" was a nice product but not a nice choise. Costs $40 each encoder and we need 3. One of them was for shooter wheel, 2 of them were for drivetrain.
Before the whole process, We had some finantial situations. Our biggest decision making parameter was the costs. We looked other encoders but we can't found any device that has low-cost and reliable for our usage.
https://docs.wpilib.org/en/stable/docs/hardware/sensors/encoders-hardware.html

Impossibilities give rise to alternative solutions:
Financial problems forced us to develop low-budget solutions. We decided to develope our own encoder. We looked at the popular methods. Firstly we looked at mouse scroll encoders but we quit this idea because the PPR(pulse per revolution) value of this kind of encoders really low. Then we thought that we can use Infrared distance sensors as encoder but then we changed this idea and we decided to use hall-effect sensor. There are some motor models include internal hall-effect based encoder modul (like NEO Brushless Motor V1.1). This kind of hall-effect sensors can detect the magnetic pole (example: MLX90393ELW-ABA-011-RE -just chip-). But we use cheaper latch-hall-effect sensor that only can detect polar shift: A-3144 DIP Hall Effect sensor. The whells perimeter was 23.5 cm, we added 8 magnet (0.5 mm) to the whell. We choosed small magnets for preventing the magnetic field overlap The tollarence is ±2.87 but this tolerance value is not increasing cumulative per rotation.

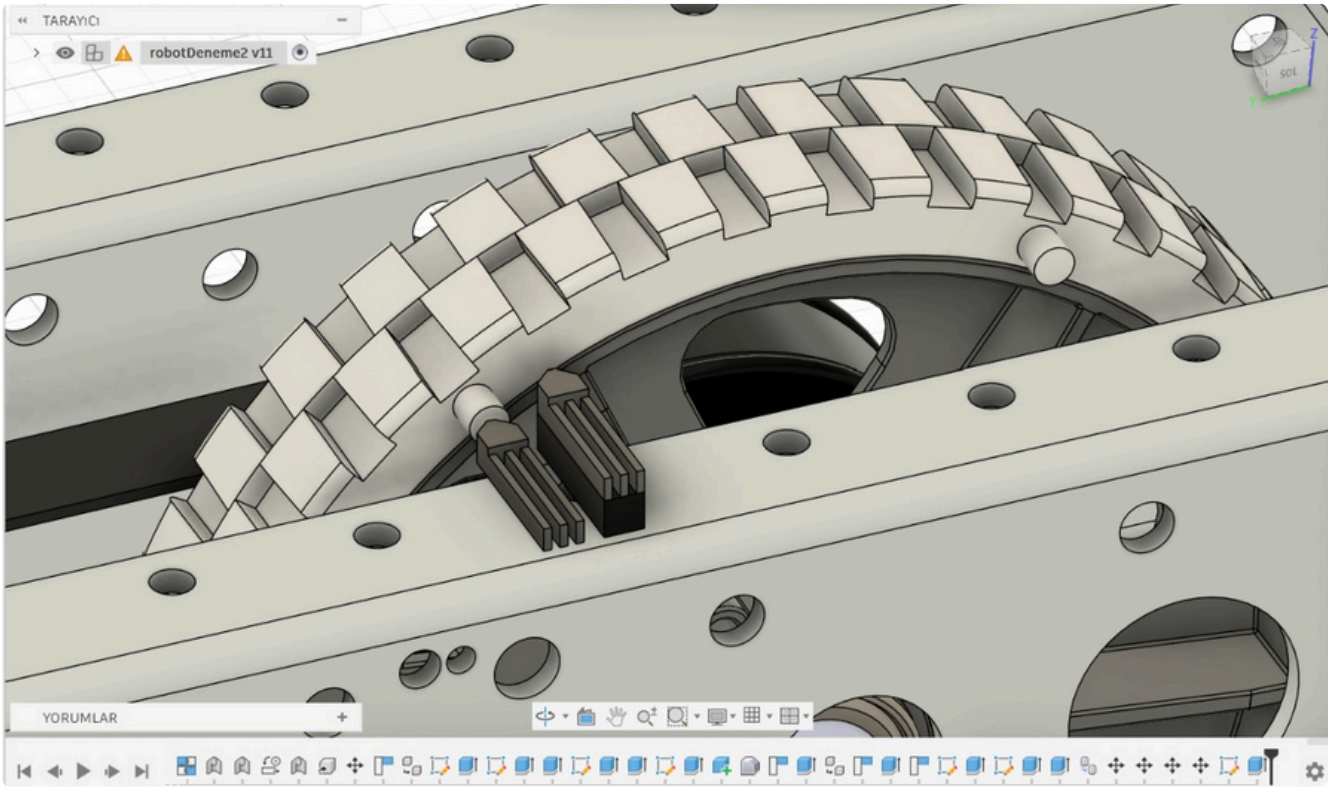
Output Rise Time	t_r	R_L = 820 Ω, C_L = 20 pF	—	0.04	2.0	µs
Output Fall Time	t_f	R_L = 820 Ω, C_L = 20 pF	—	0.18	2.0	µs

A full cycle (switching from 1 to 0 and back to 1) takes approximately 4.0 microseconds (µs) (four-millionth of a second) at the slowest rated speed.
1 second = 1.000.000
1.000.000/4 = 250.000 cycle per second
250.000 * 60 = 15.000.000 cycle per minute
That shows the sensor can handle really much cycle per second.

Referance: https://www.elecrow.com/download/A3141-2-3-4-Datasheet.pdf?srsltid=AfmBOorHUS4U97vWbjN-SnclMQqwd9cbcVmqRtKT0K8tb7d7THYw4Are

It is possible to use hall sensors as encoder. But there was a big issue, we couldn't know where we are going. The hall sensor just count the cycle and we convert it to distance. That works only we just go forward or backward. Forexample if we go 1 meter forward and return our first position, we aren't move but the program thinks we are 2 meter away from our first location. For solving th
is we can get joystick values and know where we are going but this was a unreliable method.

Solution: My teammate texted "Adding two hall sensor closely have to solve our problem" i didn' understand at first and said "It doesn't matter how much hall, they will get same pulses". Yes they gets same pulses but in different times. If spinway is clockwise, the left hall sensor gets first pulse; if spinway is opposite dirrection of clockwise, the right hall sensor gets first pulse.



After this solution, we found financial support. We change this solution which is not tested enough. We buy 3 Rotary Encoder E6B2-CWZ6C 200 Pulse.

Distance Sensor

There are some kind of distance sensors in the marketplaces. Some of the sensors are so perfect and makes its own work proper. But some of them have too cheap build quality. As our experience the cheap distance sensors are worth nothing to pay money as they have cheap build quality they have

max of 1 meters. İf you ask what is our experience in our experience. So were tought about it like 1 week to figure it out and we decided to not to use our distance sensors because you know there are many apriltags in the playground.

Software-Side:

Shooter Calculations

FRC release the field dimension. So if you know where you are in the area, you exacly know where to shoot (you know where is the hub). The Digital-Twin system, the importance of knowing where you are exactly,'s importance comes here.
https://firstfrc.blob.core.windows.net/frc2026/FieldAssets/2026-field-dimension-dwgs.pdf

Pyhsic

"Projectile motion is a form of motion where an object moves in a bilaterally symmetrical, parabolic path. The path that the object follows is called its trajectory. Projectile motion only occurs when there is one force applied at the beginning on the trajectory, after which the only interference is from gravity."
https://phys.libretexts.org/Bookshelves/University_Physics/Physics_(Boundless)/3%3A_Two-Dimensional_Kinematics/3.3%3A_Projectile_Motion

Hubs height is 72inc ≈ 182.8cm
"AndyMark part number is am-5801.
Mass is 0.448-0.S00lb (0.203-0.227kg)" (Tha balls may damage and lost its mass, because of that i take the mass value as 0.210 kg for calculations)
https://firstfrc.blob.core.windows.net/frc2026/FieldAssets/2026-field-dimension-dwgs.pdf

"The **Magnus effect** is a phenomenon that occurs when a spinning object is moving through a fluid. A lift force acts on the spinning object and its path may be deflected in a manner not present when it is not spinning. The strength and direction of the Magnus force is dependent on the speed and direction of the rotation of the object."
https://en.wikipedia.org/wiki/Magnus_effect
https://www.youtube.com/watch?v=23f1jvGUWJs

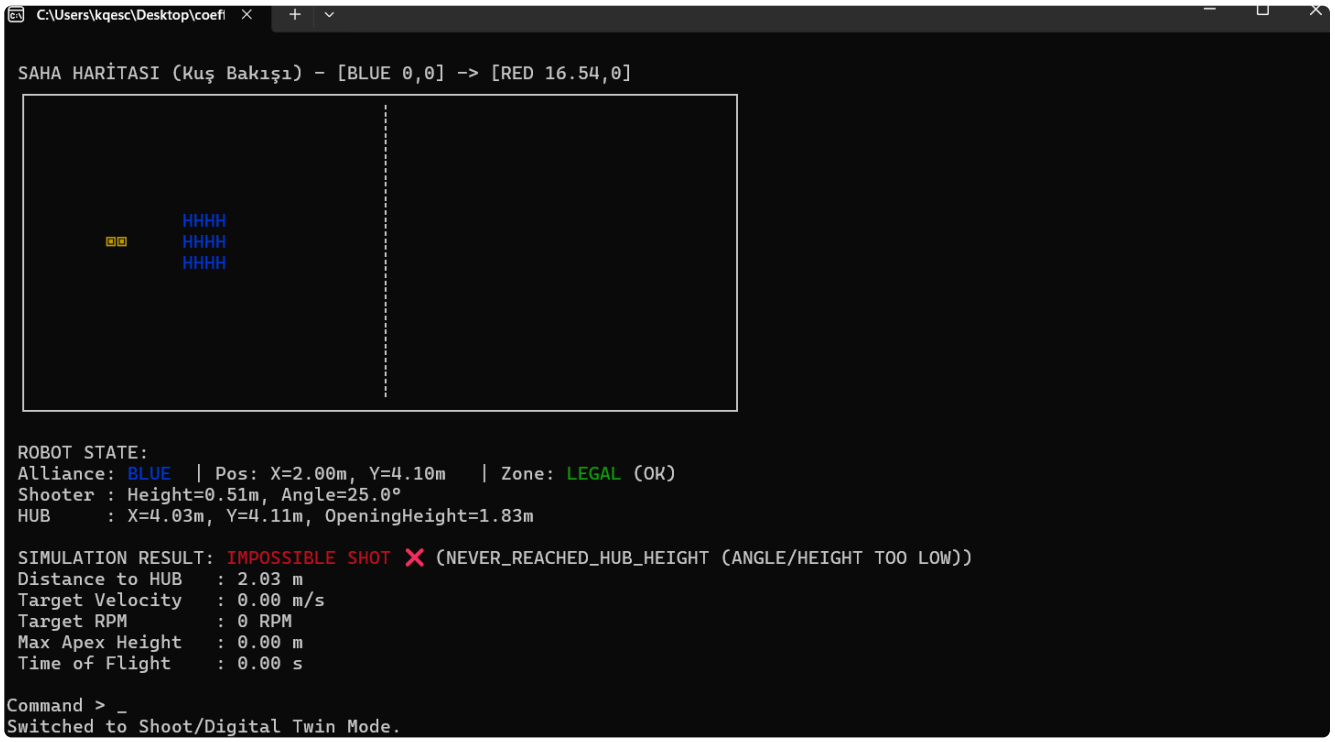
"Air resistance is the force acting on an object that is moving through air flowing in the opposite direction. The air “resists” the object’s movement, slowing it down by friction that is created as the object collides with air molecules."
https://ngpd.nikon.com/en/glossary/air-resistance.html
https://en.wikipedia.org/wiki/Drag_(physics)

We create a pysical trajertory calculator tool and we implement the real life physical as much as possible. We use this calculations in real life match.

Because of the deadline of the first release, I am I'll keep this section short. I will fully complete this section in later releases

Also we implemented this shoot calculation mode to our Terminal Dashboard software. We implement the exact same algorithm that we use in java code

https://github.com/TeknoNFL-8242/Terminal-Dashboard



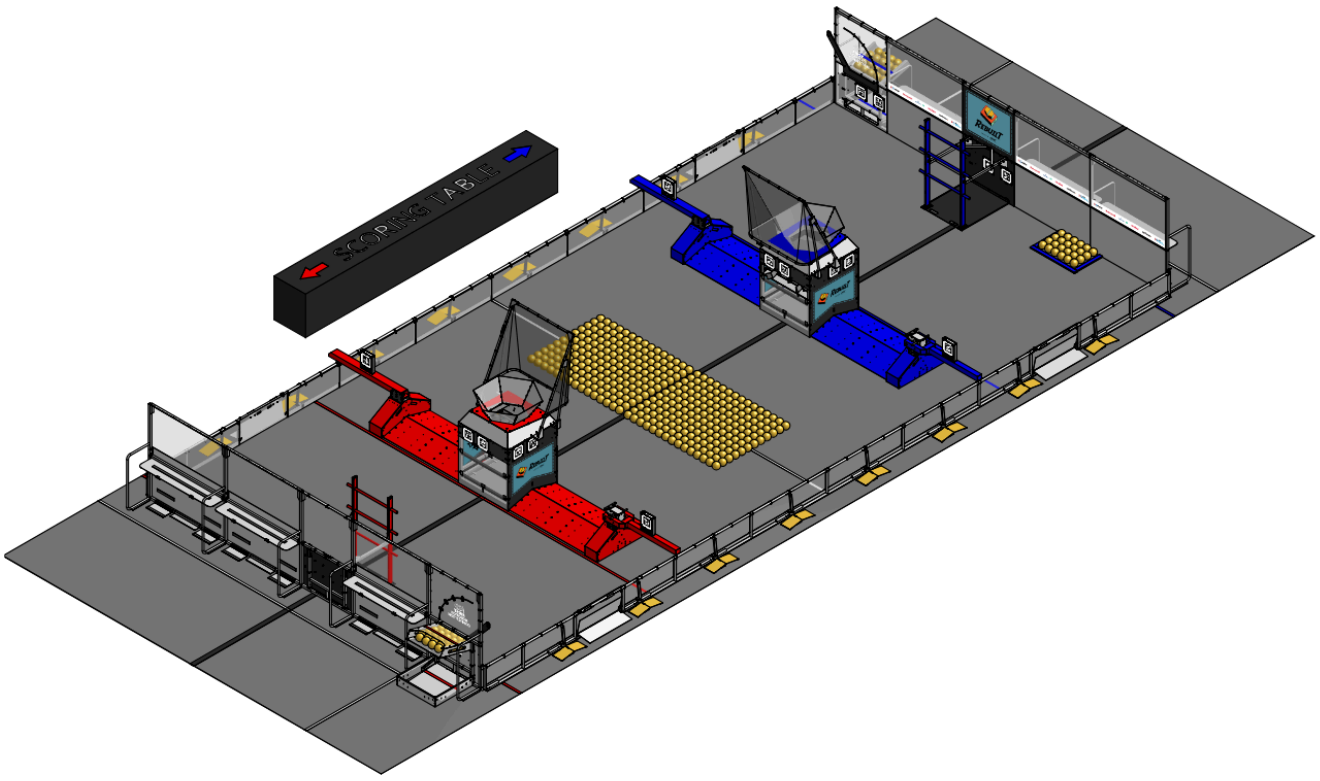
PF (Make the shooter faster and stable)

"For those not in the know, PID stands for proportional, integral, derivative control. I'll break it down:
P: if you're not where you want to be, get there.
I: if you haven't been where you want to be for a long time, get there faster
D: if you're getting close to where you want to be, slow down. Motors, in general, don't start or stop on a dime. So it's easy to overshoot, undershoot, whatever. PID allows you to dial in those three concepts with numbers"
Refferance: https://www.youtube.com/watch?v=qKy98Cbcltw
PF based on PID. Understanding PID helps tuning the flywheel shootter.

This section has been leaved blank but will complete in later

https://www.youtube.com/watch?v=fusr9eTceEo
https://www.youtube.com/watch?v=aPNCpZzCTKg&t=490s
https://docs.wpilib.org/tr/stable/docs/software/advanced-controls/introduction/tuning-flywheel.html

Autonomous Period



PID

Adding a PID feature for the Drivetrain may get some advantages. For this years spesific, weare planning to use this in.
This section has been leaved blank but will complete in later

Teleop period

This section has been leaved blank but will complete in later

FIRSTMOVE - strategy analysis framework for FRC

What is scauting (in FRC)

Scouting in FRC is basically transforming what you observe in matches into structured and usable data. It is not about “that robot looks strong” type comments. It is about measurable outputs: cycle time, scoring consistency, autonomous reliability, defensive effect, endgame success rate.

The purpose of scouting is simple: reduce uncertainty before a match. When you know how a team actually performs — not in theory, but in real matches — you can plan roles more accurately and develop counter-strategies more rationally.

Especially in seasons where robots have similar mechanical capacities, the difference is created by strategy. In that context, strong scouting becomes a force multiplier. It does not change your hardware, but it changes how effectively you use it.

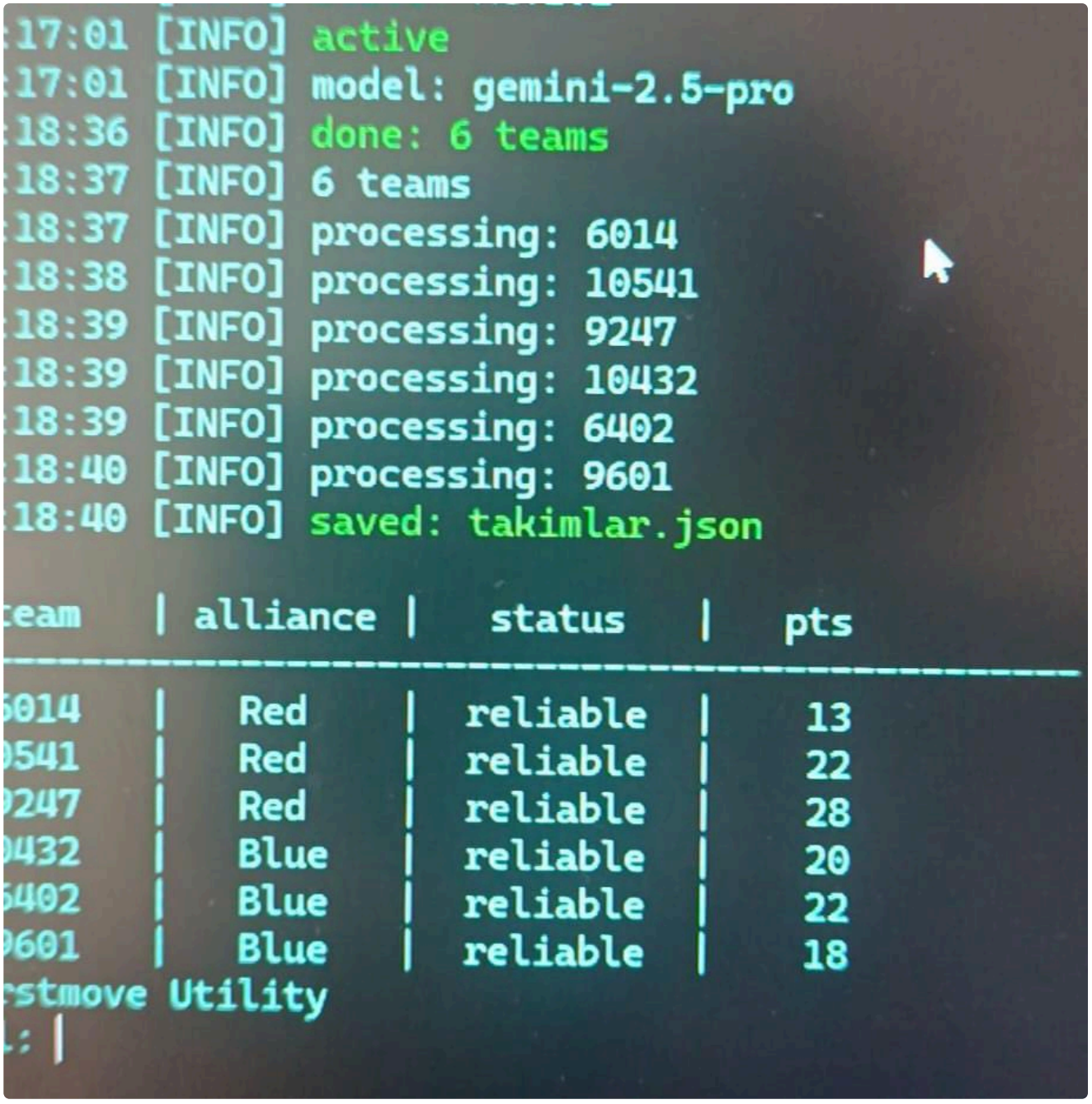
https://www.firstinspires.org/hubfs/web/program/frc/resources/intro-scouting.pdf?hsLang=en

The idea:

As a driver i have a need. Knowing my opponents' behavior pattern. Knowing behavior pattern increases my strategic capabilites. Manual scoating is a waste of human resources especially for small teams. That is wyh we created "FIRSTMOVE"

FIRSTMOVE is a strategic analysis framework for FRC that transforms raw match and team data into structured, decision-oriented insights using the M.O.V.E (Match-Oriented Value Evaluator) module. It supports alliance planning, opponent modeling, and tactical optimization. The core objective is simple: make the right first move.

I will talk about this Frameworks development but we are done for this version.



At The Competition

This section has been leav blank because we will distribute this paper in competition day. This section will be filled in and republished after the competition.

Review to This Year

This section has been leaved blank but will complete in later

NOT: This article is not yet complete; I plan to finalize it with feedback from individuals associated with FRC. You can provide feedback via the QR code.